HAUTE ÉCOLE D'INGÉNIERIE ET DE GESTION DU CANTON DE VAUD

Moodle-GitLab Integration

Bachelor Thesis

Department : TIC

Program : Computer Science and Communications Systems

Specialization : Software Engineering

Jouve Léonard

16 July 2026

Proposed and supervised by:

Delgado Pamela

HEIG-VD

Preamble

This Bachelor's thesis (hereinafter referred to as BT) is carried out at the end of the study program, with the aim of obtaining the Bachelor of Science HES-SO degree in Engineering.

As an academic work, its content, without prejudice to its value, does not engage the responsibility of either the author, the Bachelor's thesis jury, or the School.

Any use, even partial, of this BT must comply with copyright law.

HEIG-VD
The Head of the Department

Yverdon-les-bains, 16 July 2026

Authentication

I, the undersigned, Jouve Léonard, hereby declare that I have completed this work independently and have not used any sources other than those expressly cited.

Jouve Léonard

Yverdon-les-bains, 16 July 2026

Abstract

Table of Contents

List of figures

Introduction

In higher education, digital platforms play a central role in courses organization, learning materials distribution, and academic activities management. Learning Management Systems (LMS) such as Moodle are widely adopted to structure the educational experience for both teachers and students. At the same time, in computer science and related fields, source code management platforms like GitLab have become essential tools for distributing assignments, enabling collaboration, and evaluating practical work.

However, these two types of platforms, while complementary, operate independently. This separation leads to fragmented workflows: teachers must manually manage repositories, student groups, and assessment processes on GitLab, while maintaining course-related information on Moodle. For students, this results in scattered resources and information across multiple platforms, which can lead to confusion and reduce overall learning experience.

1.1. Motivation

The lack of integration between LMS platforms and Git repository hosting services introduces several challenges. Teachers often face repetitive and time-consuming tasks such as creating repositories for each student or group, managing access permissions, tracking contributions, and check submission date. These manual processes are not only inefficient but also prone to errors, especially in courses with a large number of students.

Students, on the other hand, must navigate between multiple platforms to access learning materials, enrolment information, starter code, assessment results and feedback. This fragmentation can create confusion and reduce engagement as important informations are not centralized.

Existing solutions partially address these issues but remain limited. Some are proprietary and lack flexibility, while others are tightly coupled to specific platforms such as GitHub. Moreover, they are often not deeply integrated into LMS environments, requiring additional tools or workflows that increase complexity rather than reduce it. This situation highlights the need for an open, flexible, and LMS-native solution that seamlessly integrates Git-based workflows into the educational ecosystem.

1.2. Goal

To address these challenges, this project aims to develop of an extensible integration between Moodle and GitLab. The solution consists of a custom Moodle plugin that communicates with the GitLab API. This integration will enable direct interaction with GitLab features from within the Moodle interface.

The integration will automate workflows involved in programming assignments. This includes the creation and management of repositories, automatic setup of student groups, synchronization of project data, and integration of feedback and grading mechanisms. By embedding these functionalities into Moodle, both teachers and students benefit from a unified platform where all relevant information and actions are accessible.

From an architectural perspective, the solution is designed to be modular and extensible. While initially focused on GitLab, the system will be structured in a way that allows future support for other Git repository hosting services. This ensures long-term adaptability and encourages community contributions.

Structure

The remainder of this report is organized as follows:

- **Context** introduces the different platforms and technologies involved in the project.
- **State of the Art** reviews and evaluates existing solutions related to the proposed integration.
- **Specifications** defines the project's objectives, deliverables, requirements, and expected outcomes.
- **System Architecture** describes the architectural design chosen.
- **Features** presents a detailed description of the functionalities provided by the developed integration.
- **Development Process** outlines the main stages of the project's development and implementation process.
- **Solution** details the final delivered solution.
- **Contribution and Future Work** summarizes the project's contributions, presents potential improvements and directions for future development.
- **Appendices** contain the supplementary material accompanying this report.

Context

This chapter introduces the main platforms and technologies involved in the project. The presented information aims to provide a better understanding of the overall system and the role of each component within it.

3.1. Moodle

Moodle [1] is an open-source Learning Management System (LMS) widely used by universities and educational institutions to organize courses, distribute learning materials, and manage academic activities. Its name stands for Modular Object-Oriented Dynamic Learning Environment. Moodle provides a modular structure based on courses, activities, and plugins, enabling institutions to adapt the platform according to their specific requirements.

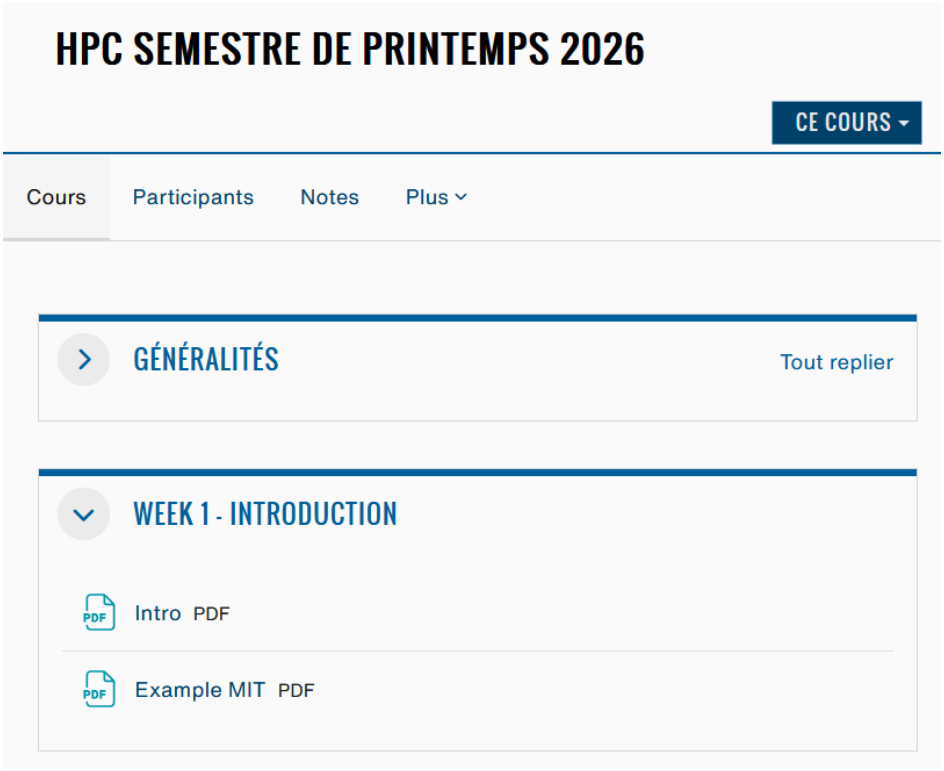

Figure 1: Moodle course

In many institutions, Moodle acts as a central platform for teaching activities, including document sharing, assessment management, and student tracking. For example, platforms like Cyberlearn (used within HES-SO) are built on top of Moodle and extend its capabilities to address institutional requirements.

As stated in the official Moodle documentation [2], one of Moodle's main strengths lies in its modular architecture. Rather than attempting to provide all functionalities required by educational institutions, Moodle focuses on being a robust Learning Management System while allowing inter-operability with external systems offering complementary services.

"There are other types of software systems that are important for educational institutions […] Generally, Moodle does not try to re-invent these areas of functionality. Instead, it tries to be the best LMS possible, and then interoperate gracefully with other systems that provide the other areas of functionality."

This extensible architecture has led to the creation of a large ecosystem of plugins and integrations, allowing Moodle to be customized for various use cases. However, implementing advanced workflows often requires specific developments adapted to the platform's architecture.

A well-known alternative to Moodle is Canvas [3]. While Moodle focuses on extensibility and customization, Canvas primarily emphasizes ease of use and straightforward setup.

The Moodle ecosystem is mainly developed using PHP [4], with JavaScript used for client-side functionalities. Therefore, the developed Moodle plugin must follow the same technological stack and adhere with the development guidelines defined by the Moodle platform.

3.2. GitLab

GitLab [5] is an open source Git repository management platform that provides tools for source code and project management and continuous integration/continuous deployment.

The main GitLab features include source code hosting and management through Git repositories, merge requests and code review workflows, issue tracking and project management tools, as well as CI/CD pipelines for automated testing and deployment. Additionally, GitLab provides extensibility capabilities through integrations based on Webhooks and REST/GraphQL APIs.

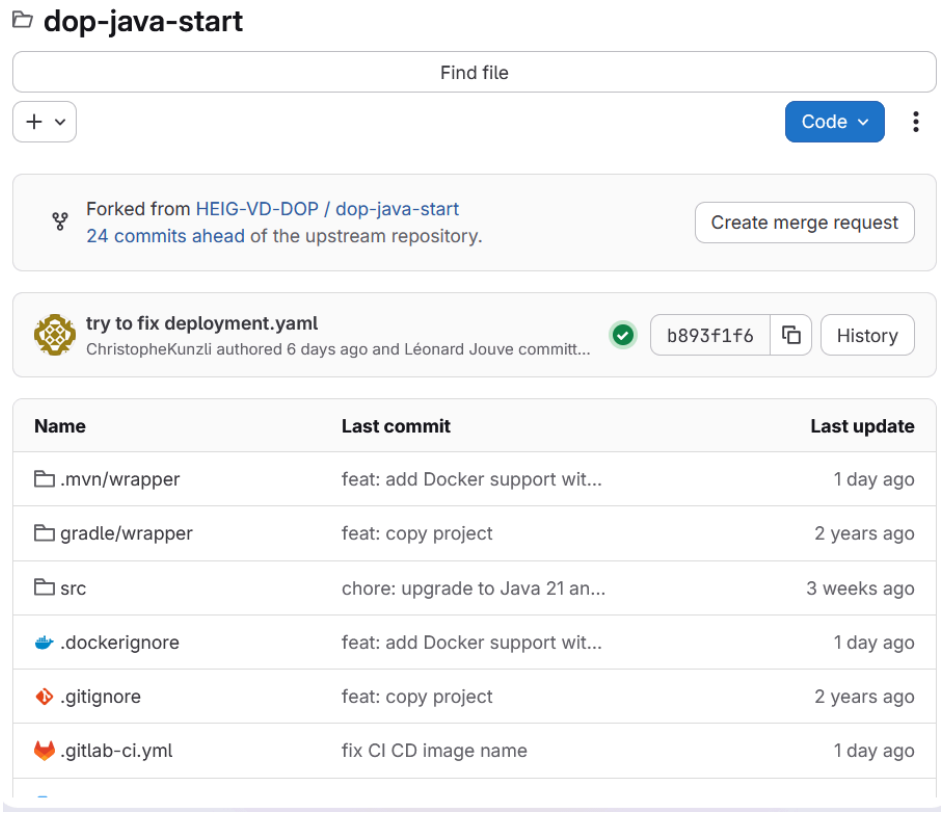

Figure 2: GitLab repository

GitLab is widely used in both industry and education due to its full DevOps lifecycle integration and strong automation capabilities. In academic contexts, it is commonly used for programming assignments, enabling students to collaborate and submit code while allowing teachers to review and evaluate contributions efficiently.

Another alternative is GitHub, a Git hosting platform that provides similar functionalities to GitLab. However, GitHub follows a different approach, relying more heavily on its developer community and

ecosystem of external integrations, while GitLab provides a more unified platform for managing the software development lifecycle.

State of the art

There are 2 faces of this project:

- GitLab interactions through REST API
- Moodle plugin creation for the UI

We must link both to have a working integration.

4.1. GitLab Client

On the GitLab API side, there a multiple existing GitLab Client solutions. A HTTP Client is an abstraction layer wrapping REST API endpoints behind a user-friendly interface.

Several implementations exist across different programming languages and ecosystems:

- GitLab Tools [6] is a Java CLI
- GitLab Haskell [7] is a Haskell library
- python-gitlab [8] is a Python package

Since the Moodle plugin will be developed in PHP GitLab PHP API Client [9] seems to be the most suitable existing implementation. It provides support for the the full GitLab API v4 specification.

However, Moodle plugins are designed to be self-contained and should avoid introducing unnecessary external dependencies. For this reason, a dedicated GitLab client will be implemented as part of this project, while taking inspiration from existing solutions and following their established design principles.

4.2. Moodle Plugin

Existing Moodle plugins provide various extensions to the platform, including additional activities, external integrations, and workflow automation. However, no widely adopted solution currently provides a complete integration between Moodle and Git repository hosting platforms.

This lack of an existing comprehensive solution motivates the development of a dedicated plugin capable of bridging Moodle with Git-based development platforms while following Moodle's plugin architecture and extension principles.

4.3. Integrations

Several existing solutions aim to bridge Learning Management Systems (LMS) with Git repository hosting platforms. These integrations mainly focus on simplifying programming assignment management, automating repository creation, and connecting educational workflows with software development tools.

4.3.1. *GitHub Classroom*

GitHub Classroom is an assignment management platform integrated with GitHub.

This solution automates the creation and setup of student repositories for teachers.

Teachers can specify an assignment name, due date, group size, starter code repository, CI jobs to run, a list of protected files to monitor for unauthorized modifications, and a feedback pull request workflow. Changes to the template repository can be synchronized across all student repositories through pull request. Reviewers are also notified when submissions are late.

GitHub Classroom is currently one of the most widely used solution in for computer science education assignments. However, its main limitation is that it is not directly integrated with the school

LMS platform. Instead, it introduces an additional layer of complexity between the LMS and the Git hosting platform.

Another drawback is that it only integrates with GitHub, which is a closed-source platform. This can lead to vendor lock-in.

The workflows that teachers will be able to automate with our plugin will be heavily inspired by this solution, and all of its main features will be available.

On 22 May 2026, GitHub Classroom published a public announcement [10] the end of their service for 28 August 2026:

"For many educators using GitHub Classroom, you may know that this product has been operating under 'maintenance mode' for the past 18 months. We know this has been frustrating, and we want to make sure Classroom gets the investment it deserves."

Following this decision, GitHub redirected users toward partner solutions:

- Codio [11]: A learning platform providing browser-based coding environments, automated grading capabilities, and LMS integration features. However, its LMS integration remains limited, and the platform is a closed-source premium solution.
- Classroom 50 [12]: A free and open-source assignment management platform developed by the Fifty Foundation. It enables instructors to distribute programming assignments, configure automated grading workflows through GitHub Actions, and manage student submissions using both web and command-line interfaces. This solution appears promising. However, it was only released on 1 July 2026, one week before this writing, and does not currently integrate directly with LMS platforms.

4.3.2. *RepoBee*

RepoBee [13] is a command-line tool that enables teachers to manage large numbers of student Git repositories across multiple version control systems. It features a powerful plugin system that allows users to either use existing plugins or develop their own. However, this solution does not integrate directly with any LMS platform.

4.3.3. *Classmoji*

Classmoji [14] is an open source LMS for teaching computer science. It is not widely adopted yet but is created with GitHub integration in the core. It claims to integrate GitHub Classroom with the LMS directly and add some additional features.

	Open source	**LMS integration**	**VCS independent**	**Extensible**
GitHub Classroom	~	×	×	×
Codio	×	~	×	×
Classroom 50	✓	×	×	~
RepoBee	✓	×	✓	✓
Classmoji	✓	✓	~	~
This work	✓	✓	~	✓

Comparison of existing solutions

✓ Supported, ~ Partially supported, × Not supported.

As shown in the comparison, existing solutions either lack deep LMS integration, rely on proprietary platforms, or are limited to GitHub. Currently, there is no open-source, Moodle-native integration

with GitLab that supports automated assignment workflows. This gap motivates the development of this project, which aims to provide an extensible, open-source solution for managing assignments through Version Control Systems directly within the LMS platform.

4.4. Academic work

4.4.1. *Canvas GitLab Integration*

Canvas GitLab Integration [15] is a research project aiming to enrich learning processes.

The paper highlights that separating version control systems (VCS) from learning management systems (LMS) leads to a fragmented and inefficient user experience, as students and instructors must switch between platforms. To address this, the authors propose a high-level programming framework that integrates Canvas and GitLab to reduce fragmentation and improve the overall educational experience.

It is a project in 3 parts:

- GitLab API
- Moodle API
- Moodle - GitLab integration API

All 3 projects are open source and written in Haskell.

The study investigated the experiences of Computer Science educators and students using Canvas and GitLab through a survey, identifying challenges caused by the lack of integration between the two platforms. The results showed that educators faced significant administrative workloads, including manually tracking student activity, linking GitLab projects with coursework, and monitoring student engagement. Students also reported difficulties associating coursework on Canvas with their GitLab projects and having to frequently switch between the two systems.

This report also mentions a key challenge in designing automated systems is balancing automation with flexibility. While automation can reduce repetitive tasks, improve efficiency, and minimize human error, excessive automation may limit users' ability to adapt workflows to specific needs or unexpected situations. Rigid automated processes can create frustration when users require control, or alternative approaches. Systems that provide too much flexibility, in the opposite may reduce the benefits of automation by increasing complexity and requiring additional manual effort. Therefore, effective system design should aim for an appropriate balance, automating labors tasks while preserving sufficient flexibility to allow users to make decisions and adjust processes.

Specifications

5.1. Objectives

This project aims to:

- Review the state of the art and identify the features and functionalities the project will include based on possibilities and limitations within the GitLab and Moodle APIs (for example: group management, project creation and feedbacks)
- Evaluate and identify most suitable architecture for the implementation
- The codebase should be modular, open to modifications and improvements
- Implement, develop and deploy the integration between Moodle and GitLab by following the DevOps principles
- Validate and test (optionally with representative users)

5.2. Deliverables

By the end of this work, we aim to have the following deliverables:

- Work report
  - Specifications document
  - Gantt chart
  - State of the art
- Work presentation
  - PowerPoint
  - Poster
- Plugin code published as open source
  - Usage documentation
  - Code tests
  - System architectural design
  - Demonstration of a working prototype

5.3. Expected result

The goal of this project is to have an extensible integration between GitLab (potentially any other Git repository) and Moodle. It would include an API automating all mentioned issues accessible from a Moodle integration plugin and thus improve teachers and students workflows.

The setup and use of the plugin should be easy and intuitive.

It should help teachers as well as students with their organization, documents / informations centralization and automate / simplify tedious workflows.

It should be released as open source to allow others to access, use, and build upon the work.

System architecture

As mentioned previously, one of Moodle's main strengths is its modularity and extensibility. It was designed around a comprehensive plugin architecture, allowing developers to extend its functionality through a wide range of plugin types. The appropriate plugin type should be selected based on the specific requirements of the desired functionality.

Some of the available plugin types include:

- **Activity modules** – Provide learning activities within courses (e.g., Forum, Quiz, Assignment).
- **Custom field** – Define custom field types used in course / profile custom fields.
- **Web service** – Implement new protocols for web service communication.
- **Assignment submission** – Provide additional methods for students to submit assignments.

For this project, the development focused on two plugin types:

- **Activity modules**
- **Custom field**

Plugins must be placed in a directory corresponding to their plugin type. The Moodle core automatically detects and registers these plugins during installation or upgrade. Once registered, their code is loaded and executed whenever the associated functionality is required.

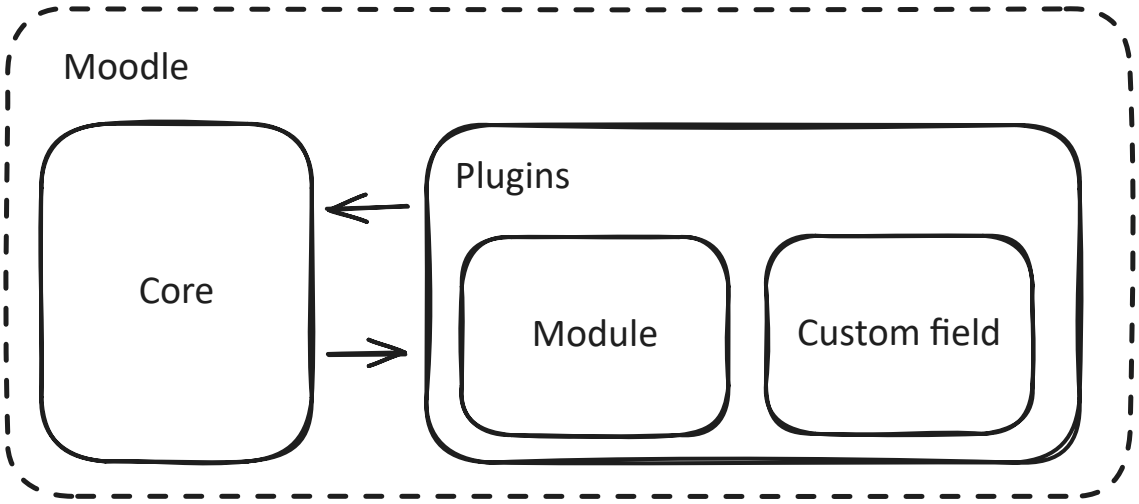

Figure 3: Moodle plugin architecture

6.1. Custom fields

The **Custom field** plugin adds a new custom field type for storing sensitive content. The field data are encrypted using Moodle builtin encryption [16] mechanism before being stored into the database and decrypted when accessed.

Custom fields are additional pieces of information that can be configured and filled in through the Moodle interface. They can be added at different levels:

- **User** level
- **Course** level
- **Group** level

Each custom field can store different types of data, including:

- **Boolean** values

- **Text**
- **Integer** values
- The newly introduced **encrypted text** type

The plugin defines two custom fields:

- A **course-level** field used to store a course-specific GitLab token.
- A **user-level** field used to store the user's GitLab username.

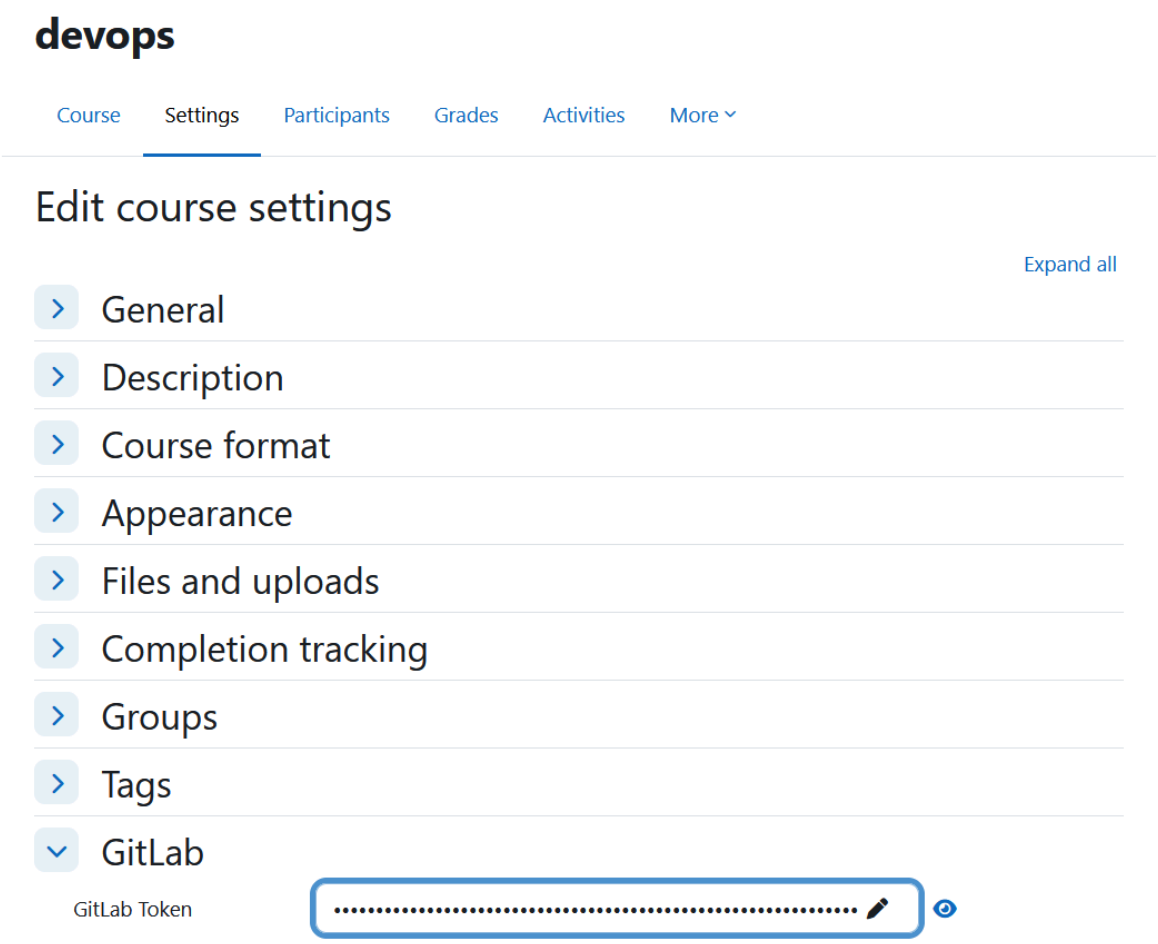

Figure 4: Course custom field for GitLab token

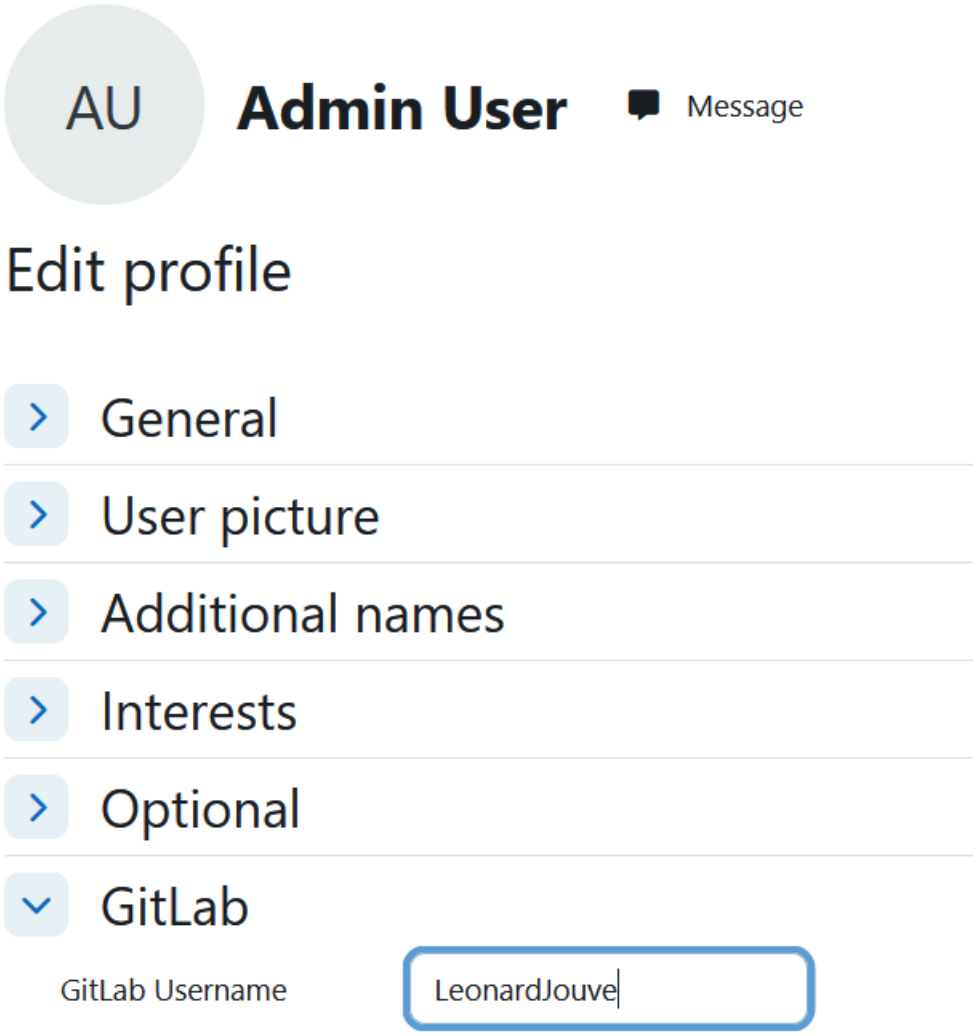

Figure 5: User custom field for GitLab username

The GitLab token field uses the newly created encrypted text field type to encrypt the token before it is stored in the database. This ensures that sensitive credentials are not stored in plaintext and cannot be directly accessed through database inspection.

The encryption is implemented using the PHP Sodium extension, which provides the XSalsa20 stream cipher for encryption and the Poly1305 message authentication code (MAC) for integrity verification. The encryption key is stored in a file external to the database, preventing exposure of the key in the event of a database compromise.

6.2. Activity module

The **Activity module** plugin introduces a new Moodle activity type called GitLab. This module is the central component of the integration between Moodle and GitLab. Creating a GitLab activity requires a GitLab token defined at the course level, as well as a GitLab username for each user who accesses the activity.

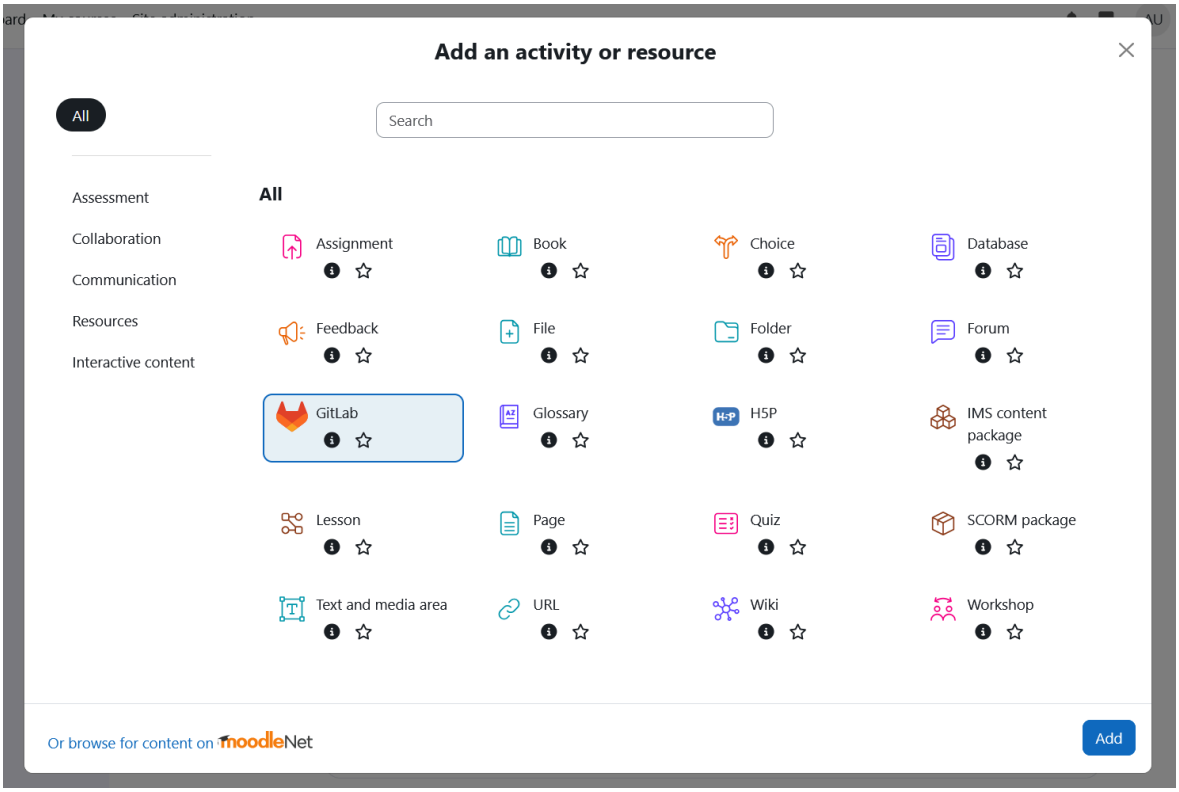

Figure 6: new **GitLab** activity module

This plugin acts as the main bridge between Moodle and GitLab. It communicates with Moodle Core to retrieve course and user data, create activity instances, and manage Moodle-side information. A dedicated GitLab Client component is responsible for all communication with GitLab.

The GitLab Client interacts with the GitLab REST API, which is used to provision and manage GitLab resources according to the desired state. Using the REST API keeps both platforms decoupled: Moodle does not need to know about GitLab's internal implementation, and GitLab does not require any Moodle-specific configuration.

This plugin acts as the central bridge between GitLab and Moodle. It communicates with Moodle Core to read course data and create activity modules while a GitLab Client subcomponent handling all communication with GitLab. This client reaches GitLab REST API, through which it provisions and manages the GitLab resources to match the desired state. The REST API keeps both platforms fully decoupled: Moodle has no direct knowledge of GitLab internals, and GitLab requires no Moodle specific configuration.

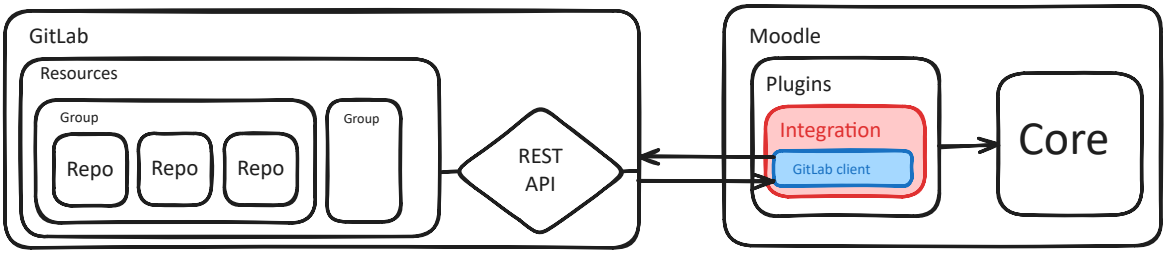

Figure 7: Moodle with GitLab integration

The plugin is divided into multiple components:

- The **Bridge** component handles the coordination between Moodle and GitLab and manages the creation of required resources on both platforms.
- The **Resources** component manages the creation and configuration of GitLab resources.

- The **GitLab Client** provides a simplified interface over the GitLab REST API endpoints.

The Bridge component delegates GitLab-related operations to the Resources component and updates the corresponding Moodle database records to maintain synchronization between both systems.

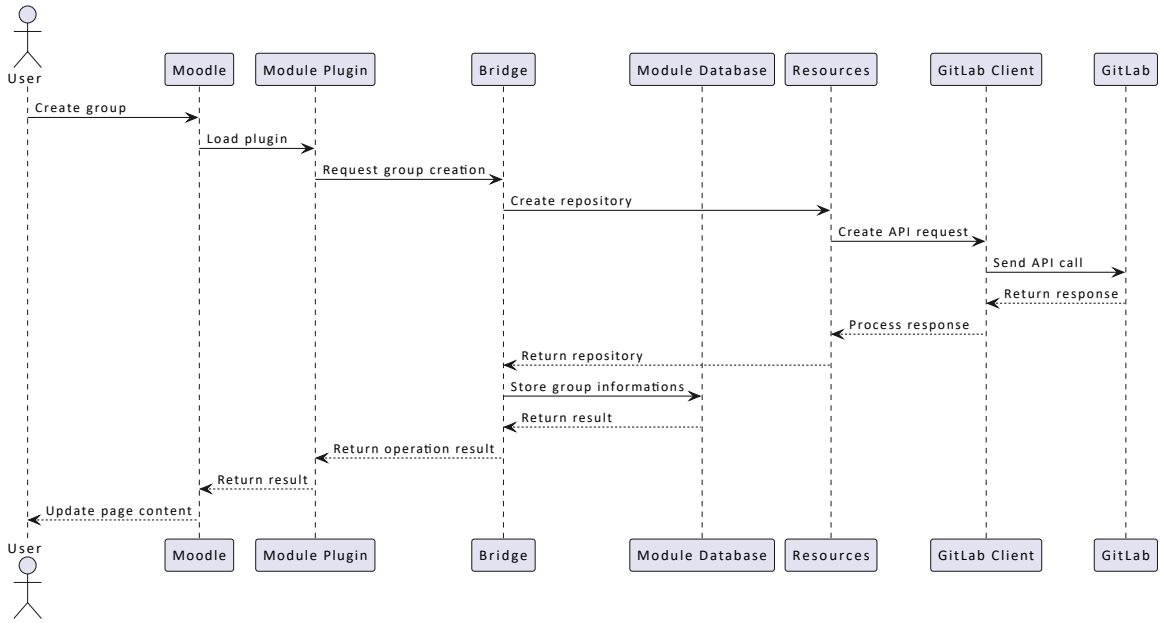

Figure 8: Components interactions during group creation

Features

The following features represent the initial requirements identified for the integration. These requirements define the expected behavior and scope of the solution, including the management of GitLab resources, the interaction between Moodle and GitLab, and the handling of user and course-specific data.

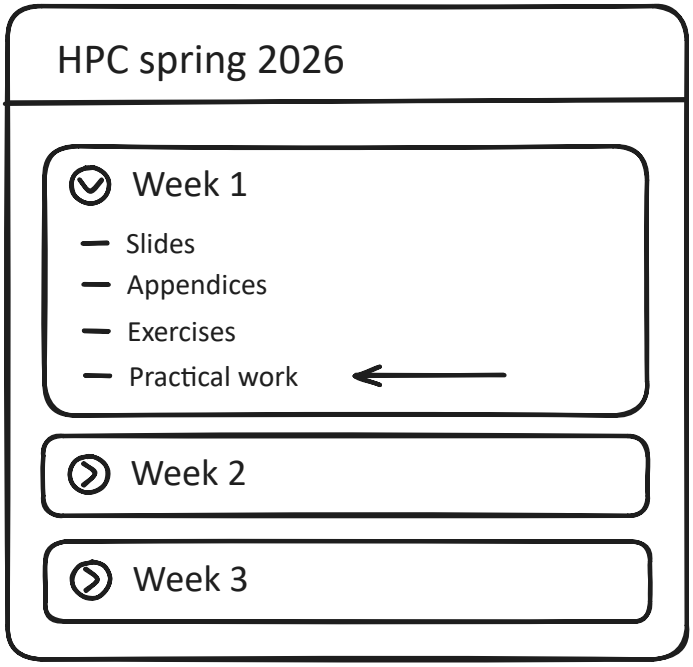

Figure 9: GitLab module on Moodle course page

7.1. Teachers

As a teacher, I want to:

- *define practical work properties such as name, description, instructions, due date, group size, reviewers*
- *create a GitLab activity module in Moodle so that I can manage a practical work linked to GitLab repositories.*
- *access a GitLab template repository so that I can prepare the initial project and solution.*
- *view all student groups so that I can monitor participation and project progress.*
- *create, edit, or remove student groups so that I can organize collaborative work.*
- *access each group repository so that I can review submissions efficiently.*
- *see the latest CI/CD pipeline results so that I can quickly identify failing projects.*
- *view individual contribution statistics so that I can assess each student's participation.*
- *download repositories source code so that I can review projects locally.*
- *know whether a group has already been graded so that I can avoid duplicate corrections.*

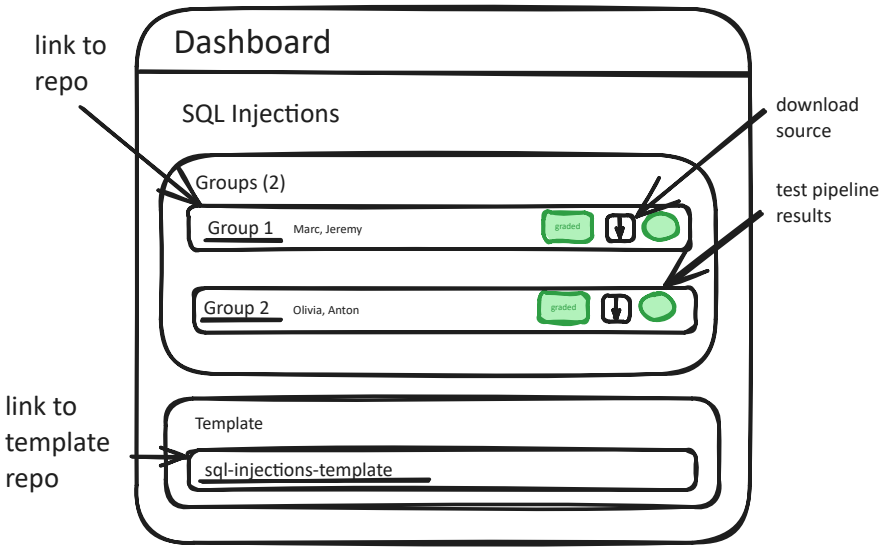

Figure 10: Teacher dashboard scaffold

7.2. Students

As a student, I want to:

- *view the list of existing groups so that I can join a team.*
- *join a group so that I can collaborate with other students.*
- *create a new group so that I can start a project team.*
- *access my group repository so that I can contribute to the practical work.*
- *view grades and feedback on Moodle and GitLab merge requests so that I can understand my evaluation.*
- *receive a notification when grading is completed so that I know when feedback is available.*
- *see the assignment due date in the Moodle calendar so that I can manage my schedule.*

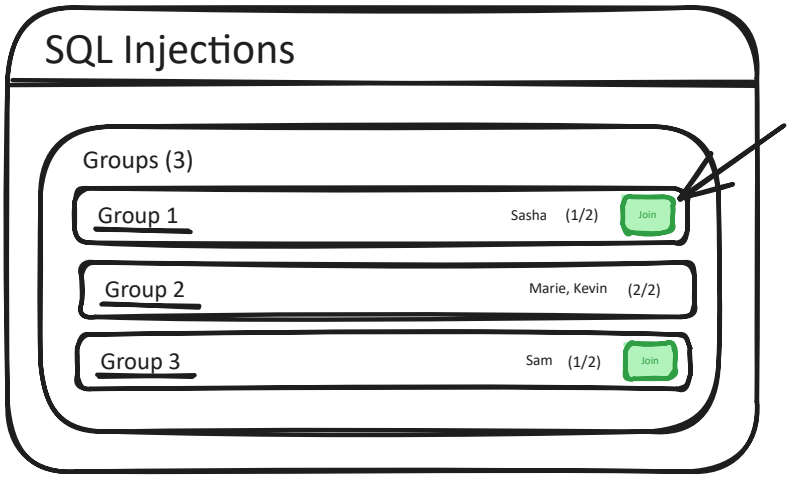

Figure 11: Student groups list scaffold

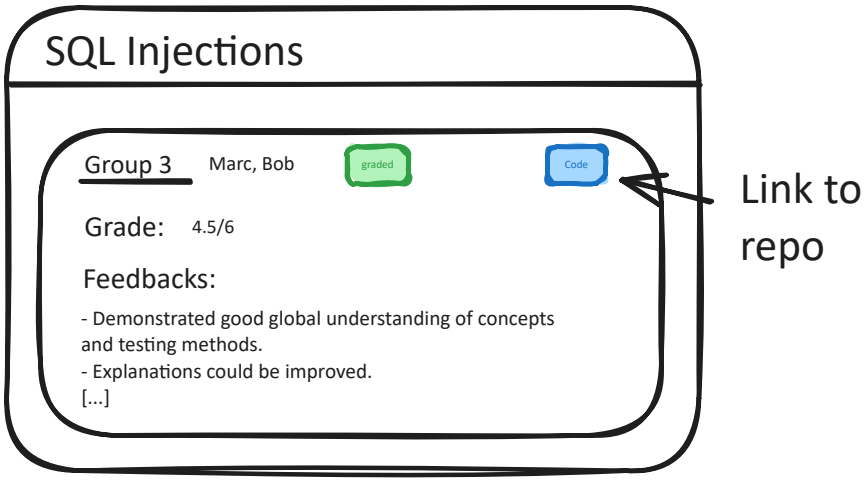

Figure 12: Student group scaffold

7.3. Automation

As a teacher, I want to:

- *create a GitLab group automatically when a Moodle activity is created so that setup is simplified.*
- *create a template repository automatically so that all student repositories share the same base project.*
- *see updates to the template repository propagate to student repositories through merge requests so that projects stay synchronized.*

As a student, I want to:

- *have a repository created automatically when my group is formed so that I can start working immediately.*
- *see the practical work instructions to be automatically added as GitLab issues so that I can easily track requirements.*
- *have a solution branch to become available after the deadline so that I can compare my work with the expected solution.*

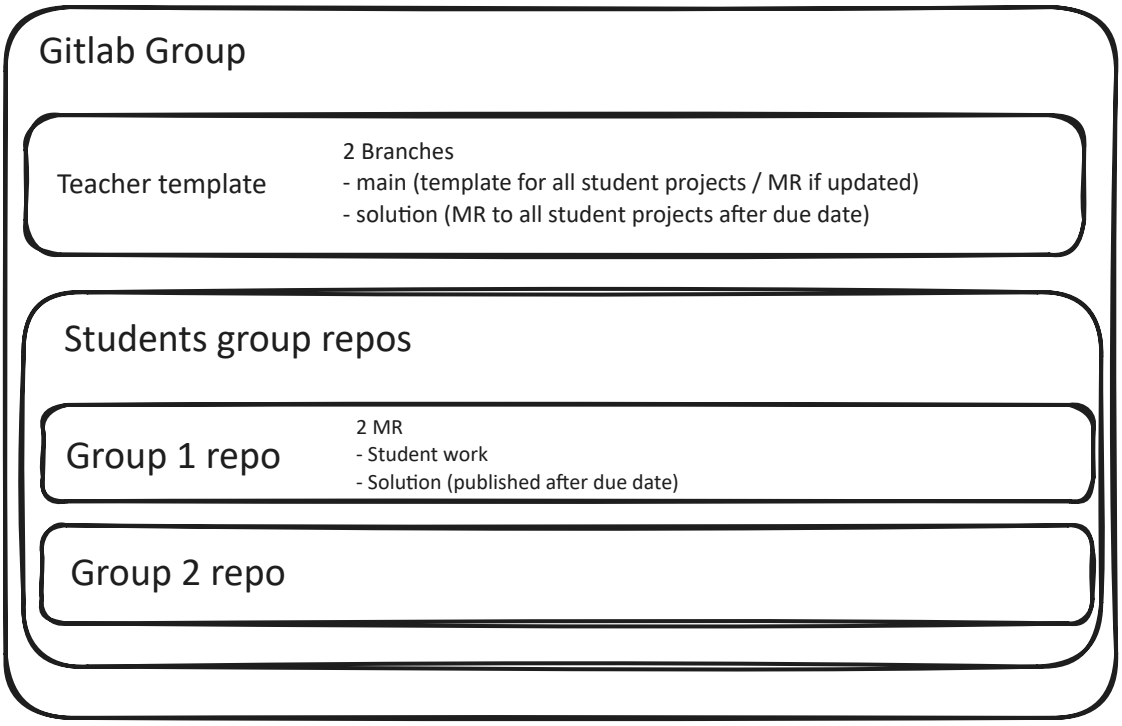

Figure 13: GitLab practical work structure

7.4. Other

Some custom integrations could be implemented with external tools, but these have been defined as out of scope for this project and could be added later through separate Moodle plugins.

Potential candidates include assessment platforms such as Gradescope [17] and ANS [18]. However, integrating with these proprietary platforms would introduce dependencies on third-party services and may not suit all teaching workflows, as instructors often have different preferences.

Development Process

This project was carried out over the course of one academic semester, with a total workload of 450 hours dedicated to the different activities required to complete the project. These activities included the initial research phase, specification and design of the solution, implementation and testing, as well as the documentation. This chapter presents the main stages of the development process and describes the progression of the work throughout the project.

8.1. Research and Specification

The first months were mainly focused on establishing the requirements, the planning and the feasible features for the project.

I came out of this with a well defined feature list, Gantt chart Section 11.1 and Kanban board Section 11.5.

8.2. Workflow

The development workflow defined for this project follows a Git feature branch strategy to ensure isolated, organized, and traceable changes. Each feature is implemented in a separate Git branch, allowing for review before integration into the main codebase. The task management is handled through a Kanban board [19]. Items are organized by priority and annotated with their dependencies. This ensures critical tasks are addressed first and visibility over blocking / critical tasks.

8.3. Proof Of Concept

I then realized as a proof of concept [20] a simple GitLab integration within Moodle. I created a Moodle module plugin. This kind of plugin adds a new type of resources a teacher can add to a course.

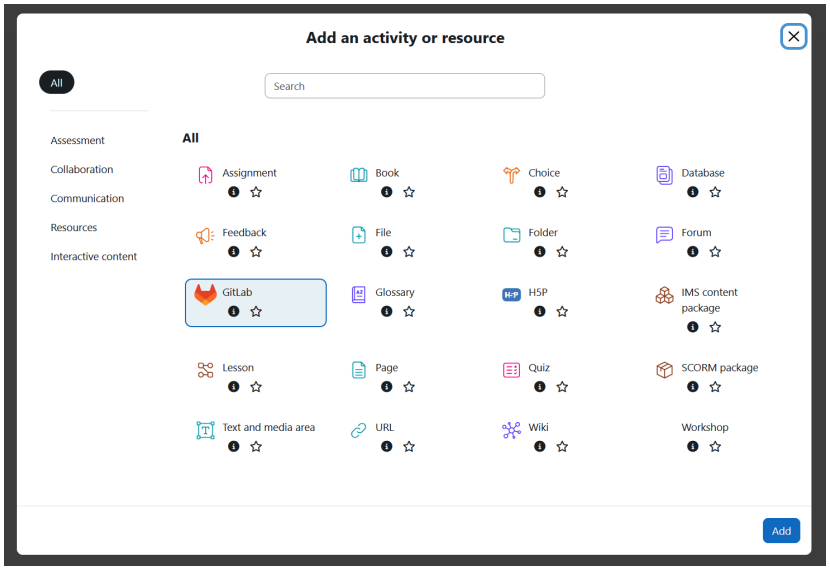

Figure 14: Modal for adding resources has a new **GitLab** type of module

The teacher must provide a GitLab token at the resource creation time.

Once created, the students can open the module created and view a button which triggers the creating of a GitLab repository. They can also see a list of existing repositories in the GitLab group with a direct link to them.

Figure 15: GitLab module view

Figure 16: Resulting repository created on the selected GitLab group

8.4. Reproducibility

To provide a full featured reproducible development and testing environment, I added Continuous Deployment [21].

To provide a reproducible and automated deployment environment, an Infrastructure as Code approach was adopted using Terraform [22]. Terraform uses a declarative configuration model, allowing the desired state of the infrastructure to be described rather than specifying the sequence of operations required to achieve it. It was used to define and provision the required cloud infrastructure, including the creation and configuration of an AWS EC2 instance bound to an Elastic IP.

Once the infrastructure was provisioned, Configuration as Code principles were applied using Ansible [23]. Ansible playbooks were created to automate the installation and configuration of the software environment. This approach ensures that the deployment process is repeatable and maintainable.

The deployed environment consists of a Moodle instance protected by Authentik [24] as an identity provider. Authentik was selected as it provides an open-source authentication solution supporting modern authentication protocols and I was already familiar with it, reducing the complexity.

To handle communication between the different services, Traefik [25] was used as a reverse proxy and ingress controller. Traefik is particularly suited for containerized environments due to its ease of use and ability to dynamically configure routing rules based on running services.

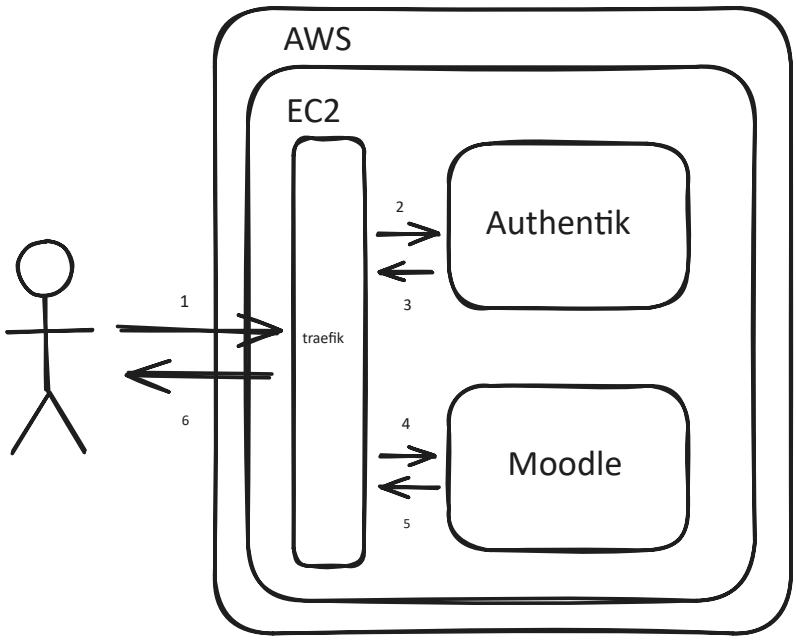

Figure 17: IaC architecture

I finally added a GitHub action to automate the updates after each push on the main branch of the project.

8.5. Solution design

The solution was designed around a modular architecture separating Moodle-specific logic, GitLab resource management, and API communication. It follows the Moodle Activity module plugin structure and naming conventions described in the official documentation.

The main design goals were to keep Moodle and GitLab independent through the use of the GitLab REST API, centralize GitLab communication inside a dedicated client component, and follow, as much as possible, the design principles and coding practices used within the Moodle codebase.

8.6. GitLab client

The GitLab client component provides an abstraction layer over the GitLab REST API. It centralizes all communication with GitLab and exposes high-level methods for interacting with GitLab resources.

This abstraction isolates GitLab-specific implementation details from the rest of the plugin. It simplifies resource management, improves maintainability, and allows future changes to the GitLab API communication layer to be handled independently from the Moodle integration logic.

The client manages authentication by attaching the given GitLab token to each request and provides access to dedicated components responsible for different GitLab resource types.

The design of the GitLab client was inspired by the structure and usage patterns of the previously introduced GitLab PHP API Client [9] library.

The `Gitlab` class acts as the main entry point and exposes dedicated objects for different GitLab resource types:

- `group()` handles GitLab groups
- `project()` handles repositories/projects
- `user()` handles users

- `branch()` handles branches
- `member()` handles permissions and memberships
- `issue()` handles issues
- `merge_request()` handles merge requests
- `pipeline()` handles CI/CD pipelines
- `webhook()` handles webhooks
- `file()` handles repository files

This means that other parts of the Moodle plugin can write code like:

```
$client->group()->create($name, $parent_id, $extra);
$client->project()->fork($repository_id, $name, $group_id, $extra);
```

8.7. Module creation

The activity module implementation introduces the GitLab activity type into Moodle. The module stores the configuration required for each activity instance, including the name, description, selected GitLab parent group, maximum group size, reviewers, submission due date.

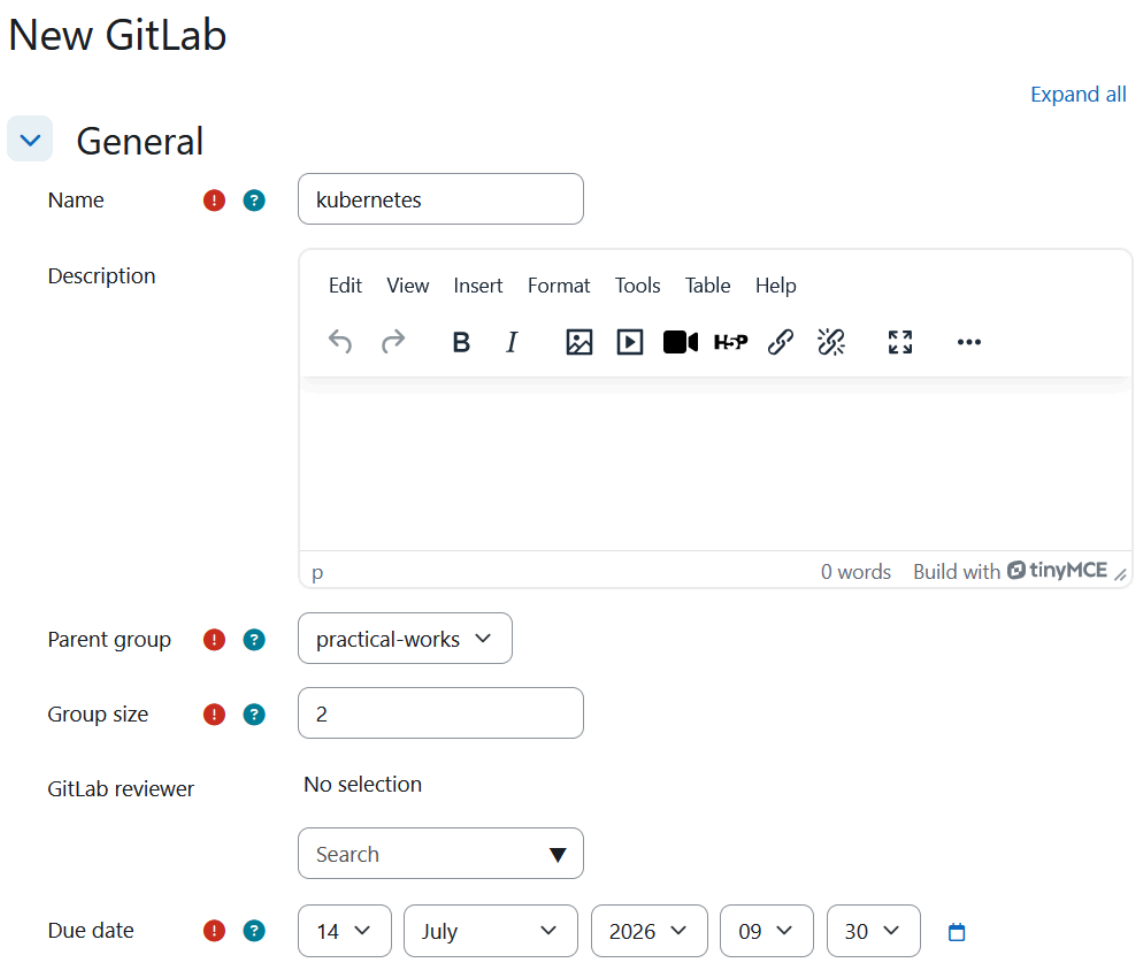

Figure 18: Configure GitLab activity module

8.8. Custom fields

Custom fields were implemented to store additional information required by the integration within the appropriate Moodle contexts.

Two custom fields were introduced:

- A course-level field storing the GitLab token used to manage resources for a specific course.
- A user-level field storing the GitLab username associated with a Moodle user.

The integration links Moodle users with their GitLab identities through the configured GitLab username field.

This association allows the plugin to automatically assign GitLab permissions when creating repositories and groups.

Repository permissions are configured according to user roles:

- Students receive access to their group repository with the **developer** access level.
- Teachers and reviewers receive the **maintainer access level**.

8.9. Database design

The plugin uses dedicated database tables to store the relationships between Moodle activities and GitLab resources.

The following tables contain the data required for the plugin operation:

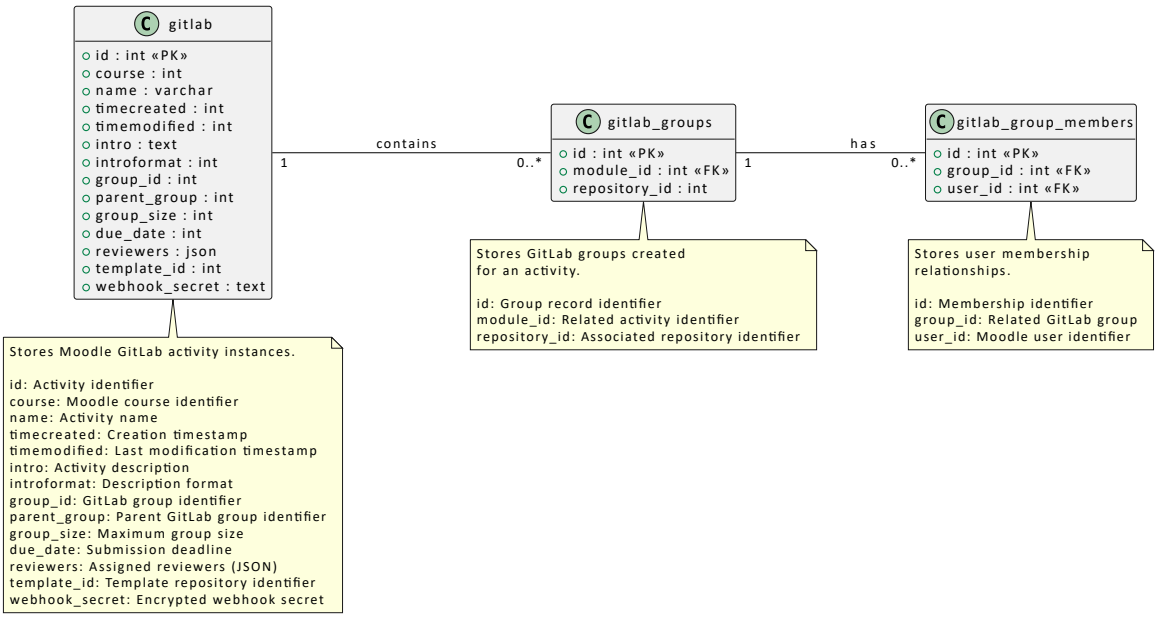

Figure 19: Module plugin database schema

8.10. Teacher dashboard

The teacher interface provides access to the main activity settings, including the module name and description, the GitLab parent group where resources are created, the assigned reviewers, and the submission due date. All deadlines are interpreted using the configured Moodle instance timezone to ensure consistency.

Teachers and reviewers have direct access to the GitLab template repository associated with the activity. This repository acts as a central workspace for preparing practical work by defining the initial project structure, instructions, and required resources before group repositories are generated.

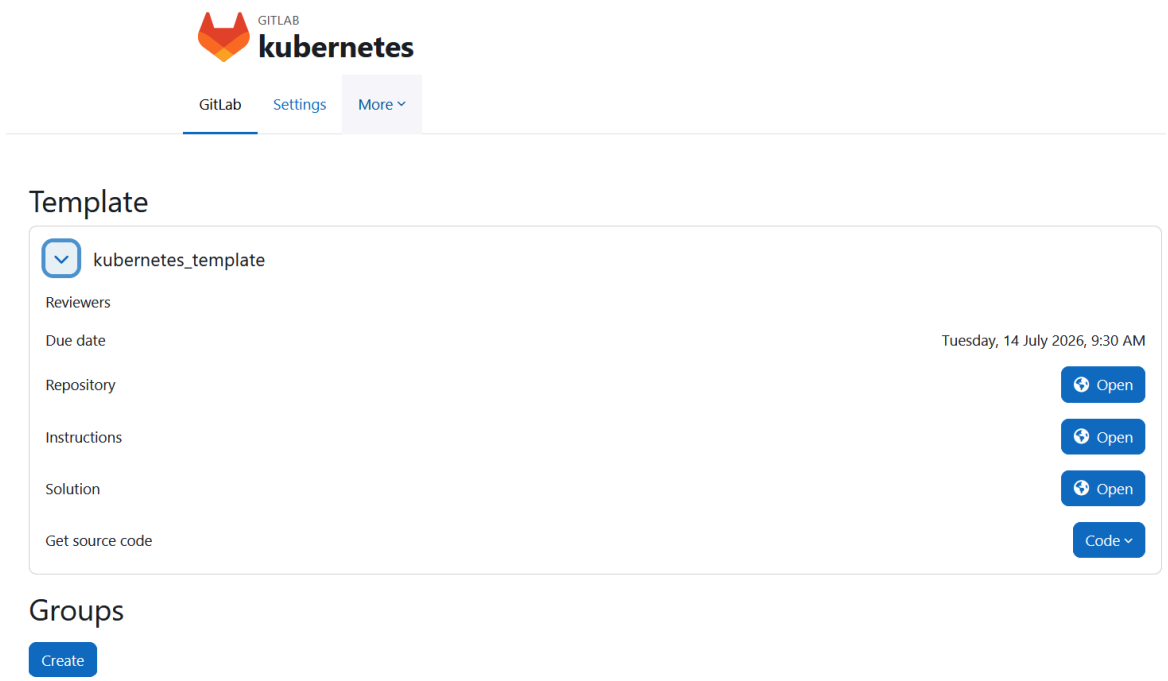

Figure 20: Teacher dashboard template repository

The interface also provides an overview of the student groups associated with the activity. Teachers can create and manage groups, assign members, and access the corresponding GitLab repositories and submission information.

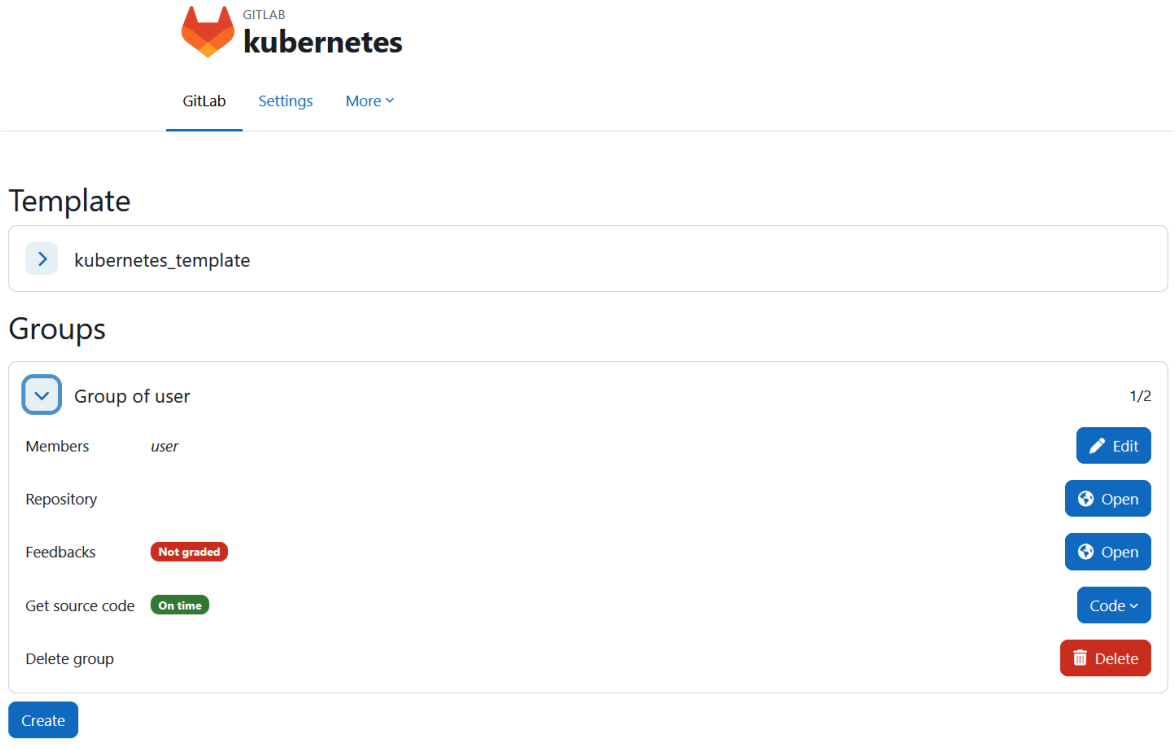

Figure 21: Teacher dashboard's student groups list

For evaluation purposes, reviewers and teachers can retrieve group submissions using different methods. They can clone repositories using SSH or HTTPS, inspect the exact repository state at the submission deadline by checking out the corresponding commit, review submissions through merge requests centralized in the template repository, or download the source code as a ZIP archive.

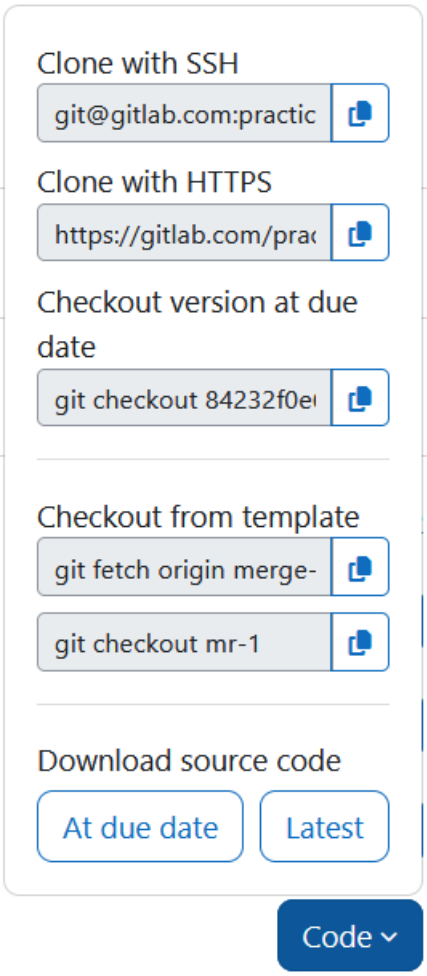

Figure 22: Different ways to retrieve group code

8.11. Student view

The student interface provides access to GitLab-related information directly from Moodle.

Students who are not assigned to a group can view available groups, create a new group, or join an existing group that has not reached its maximum capacity.

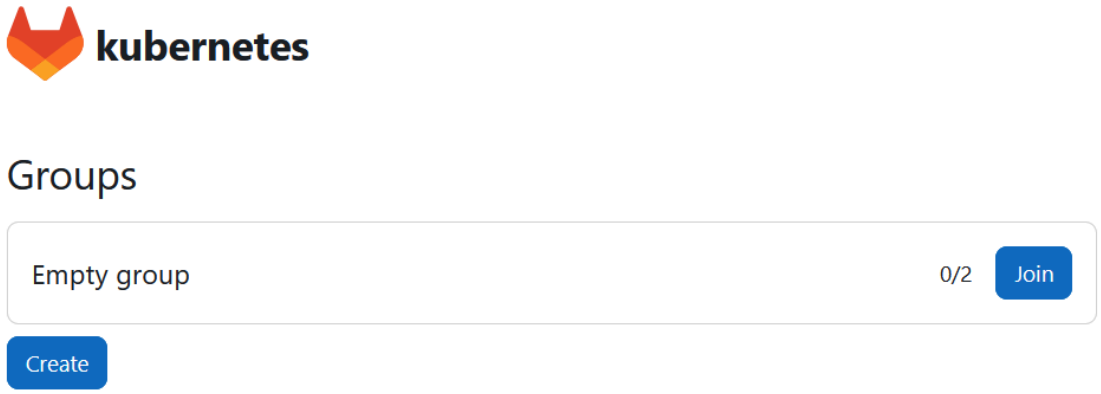

Figure 23: Student group selection interface

Once a student becomes a member of a group, additional options become available, including access to the group's GitLab repository and submission-related information.

The integration allows students to follow the complete submission workflow from Moodle without requiring them to manually switch between Moodle and GitLab.

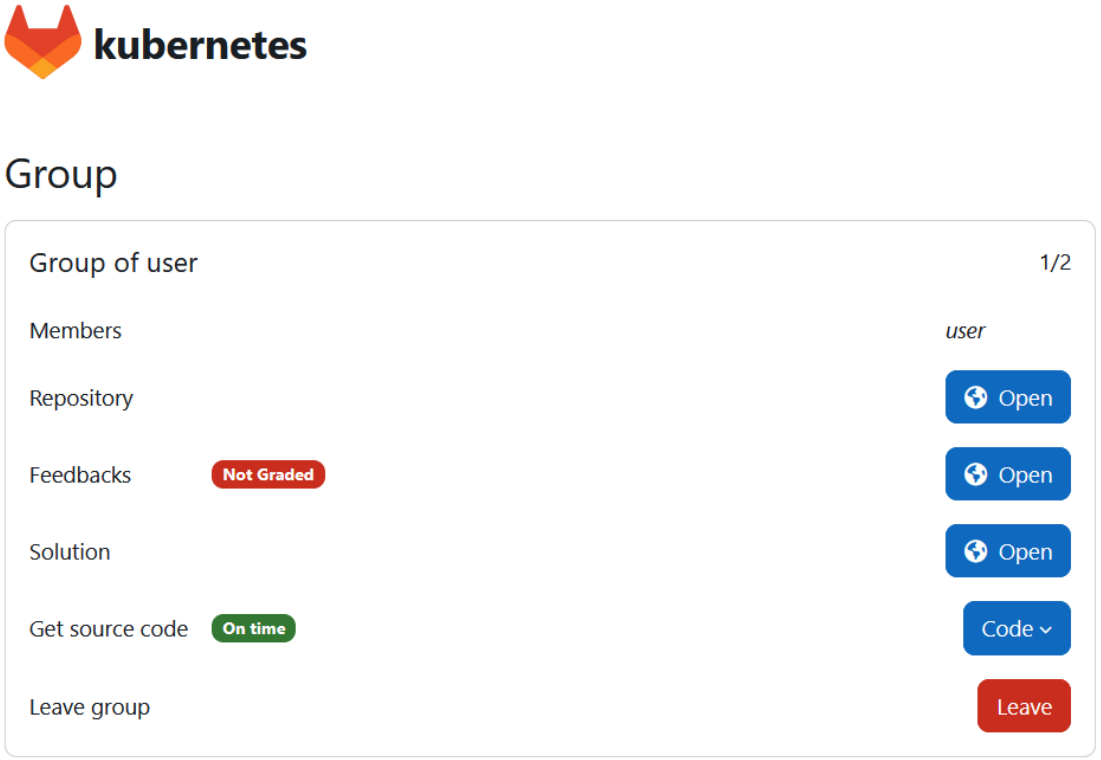

Figure 24: Student group interface

The interface also provides information about the submission status of each group. Students can identify whether their work has been graded, submitted on time, or submitted after the deadline.

After the submission deadline, students can also access the solution provided by teachers directly from the Moodle interface.

8.12. Automate GitLab workflows

The plugin automates the creation and management of GitLab resources required for practical assignments.

For each activity, the plugin automatically creates a dedicated GitLab group inside the selected parent group. This group acts as a container for all resources associated with the Moodle activity.

A template repository is also created for the practical work. This repository is accessible by the GitLab token owner and the selected reviewers and provides the initial project structure used to generate student repositories. It contains the practical work instructions as an issue, which is replicated to each group repository, as well as two branches: the `main` branch used as the template for initializing group repositories and the `solution` branch containing the reference solution. After the submission deadline, the solution branch is proposed to each group through a merge request.

For each student group, the plugin creates a fork of the template repository and configures access permissions according to the user's role. Group members receive Developer access, the GitLab token owner receives Owner access, and selected reviewers receive Maintainer access.

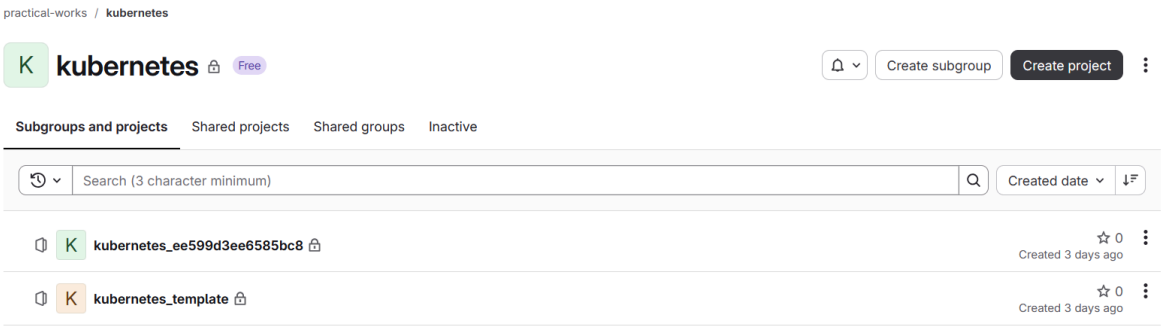

Figure 25: GitLab created group containing the template repository and a group repository

Each group repository contains a copy of the practical work instructions issue and two branches: the `main` branch used by students to submit their work and the `base` branch used as a protected baseline for tracking modifications. A merge request from `main` to `base` is automatically created to simplify the review process.

After the submission deadline, each group submission merge request is also mirrored to the template repository, allowing reviewers to access all submissions from a single location.

This automation removes repetitive manual configuration, ensures consistency across assignments, and provides a standardized workflow for managing practical work.

8.13. Bind Moodle user interface with GitLab

In the previous steps, links between Moodle and GitLab resources were represented using placeholders. The goal of this step was to replace these placeholder values with the actual links corresponding to the GitLab resources created during the workflow.

8.14. Notifications, Calendar events and Adhoc Tasks

The plugin integrates with Moodle notification and calendar systems to provide deadline reminders and keep users informed about relevant events.

Submission due dates are added to the Moodle calendar, allowing students to track upcoming deadlines directly from the Moodle interface.

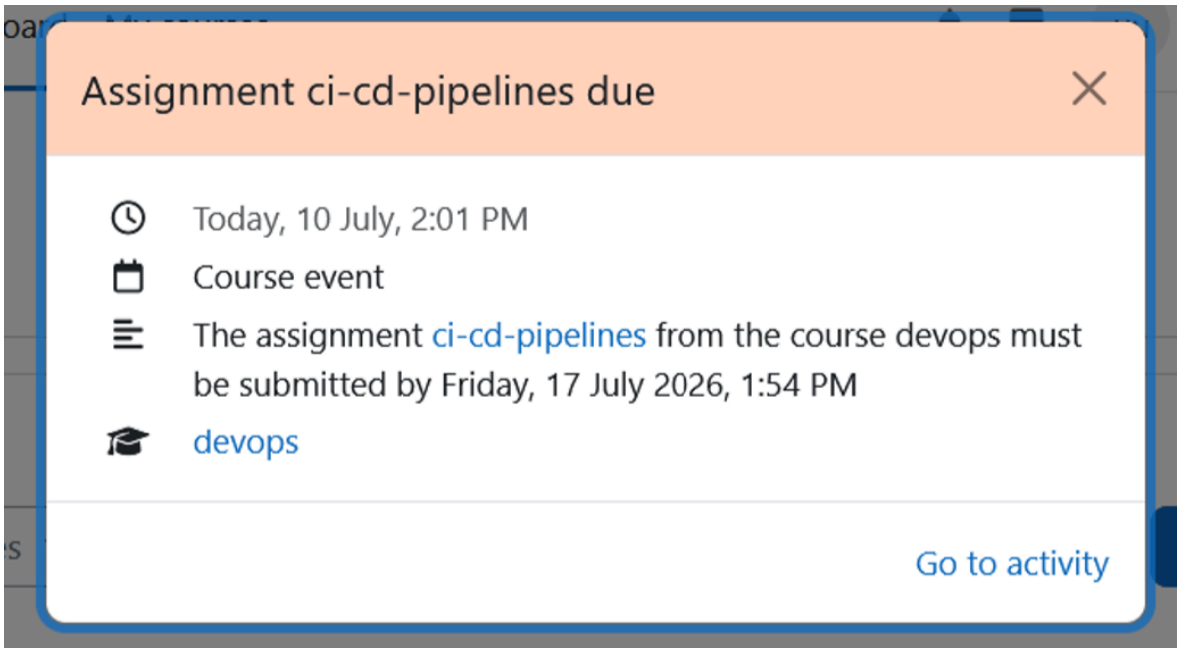

Figure 26: A Moodle submission calendar event

Notifications are generated for different events, including:

- One day before the submission deadline.
- The submission deadline date.
- Submission evaluation completion after the submission merge request is closed.

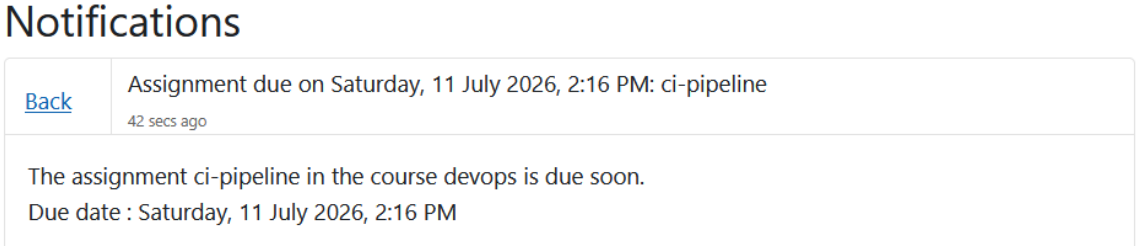

Figure 27: A Moodle submission reminder notification

The plugin also uses Moodle Core's Adhoc Tasks mechanism to execute asynchronous operations. This allows developers to schedule tasks for later execution while Moodle manages their execution through the `cron.php` script.

The cron.php script is a required component of any Moodle instance and must be executed regularly using a system cron job, as described in the official documentation [26].

8.15. Webhooks

Webhooks were implemented to synchronize GitLab events with Moodle and keep both platforms consistent. The are automatically created and configured in the repositories provisioned by the plugin.

GitLab sends signed webhook requests to the Moodle endpoint:

```
https://<moodle-host>/mod/gitlab/webhook.php
```

To ensure the authenticity and integrity of webhook requests, GitLab signs each payload using a module-specific secret generated by Moodle. This secret is stored encrypted in the Moodle database

using the same encryption method as for the GitLab tokens. It is used by the plugin to verify each incoming webhook request before processing the event.

The webhook endpoint must be publicly accessible without any authentication layer on top, as requests are sent directly by GitLab. If authentication cannot be disabled, additional authentication information can be configured through custom headers manually in the GitLab webhook settings.

Two webhook types are used: template repository webhooks and group repository webhooks.

A webhook is automatically created for the template repository. It listens for issue events and push events affecting the `main` branch. When one of these events occurs, GitLab notifies Moodle that the template repository has been updated. Moodle then propagates these changes to all group repositories. Updates to the practical work instructions issue are applied directly to the corresponding issue in each group repository, while changes pushed to the `main` branch are propagated through merge requests, allowing groups to incorporate the latest updates. This mechanism allows teachers and reviewers to modify the practical work even after student repositories have been created.

Figure 28: GitLab template repository webhook

A webhook is also created for each group repository to monitor submission merge request events. It listens for close actions on submission merge requests. When a merge request is closed, GitLab notifies Moodle, which identifies the corresponding group submission and updates the submission status. This allows grading-related events to be reflected automatically within Moodle and enables students to be informed when their work has been evaluated.

8.16. Protected files

A GitLab CI verification job is automatically created in every repository to detect modifications to protected files.

The job runs during merge request pipelines and compares the files modified by the merge request against a list of protected file patterns.

The `.gitlab/protected-files` file contains one protected file or pattern per line, allowing teachers to define which files should not be modified as part of the assignment.

If a modified file matches one of the protected patterns, the pipeline fails and reports the affected file. This provides immediate feedback to students when they modify a protected file and helps reviewers identify changes that may require additional attention during the evaluation process.

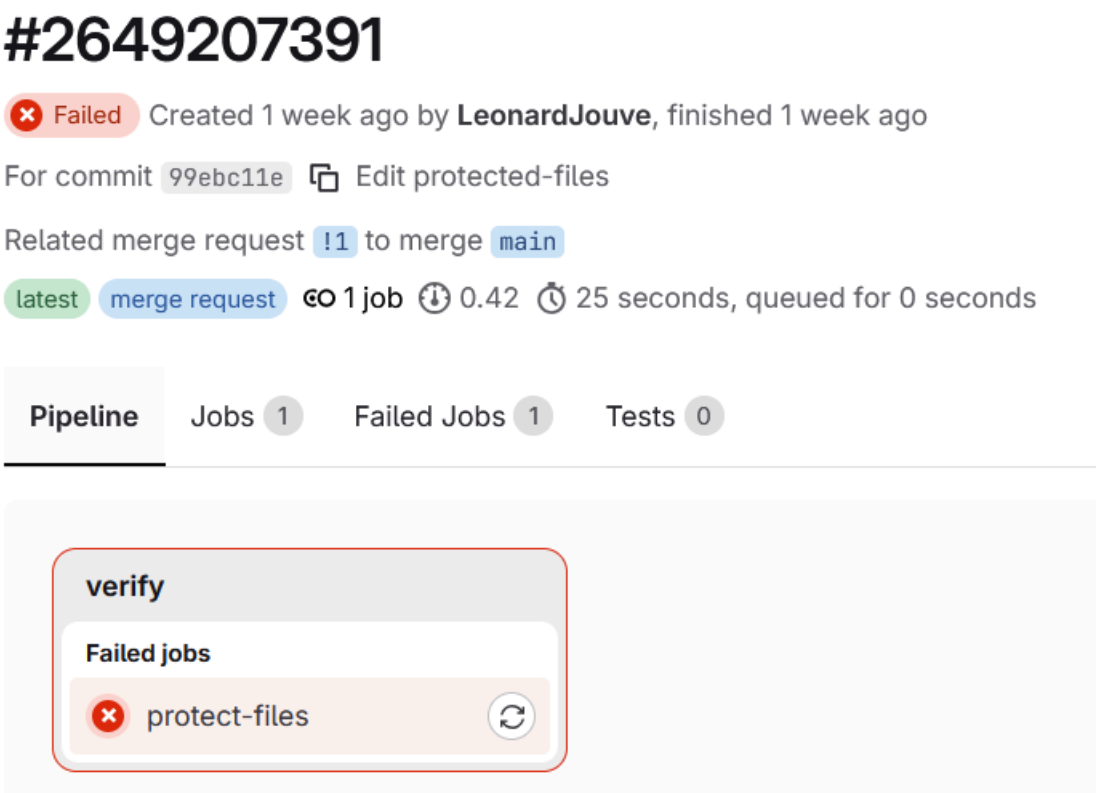

Figure 29: GitLab CI protected files pipeline

This mechanism is intended as an informational safeguard rather than a security feature. It assists reviewers in identifying potentially sensitive modifications but does not prevent users from modifying the CI configuration itself.

8.17. Custom GitLab host

The integration supports custom GitLab host by adding a configuration to specify a GitLab URL different from the default GitLab instance. The configured host is used by the GitLab client when communicating with the REST API, allowing the plugin to work with self-hosted GitLab deployments.

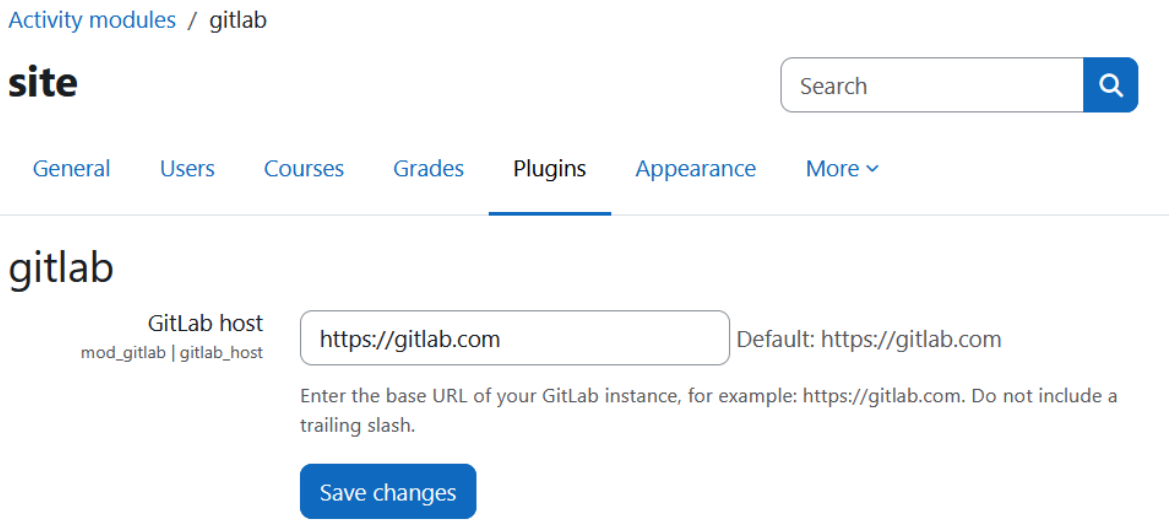

Figure 30: Plugin custom GitLab host configuration

Solution

This chapter presents the final solution delivered by the project. It summarizes the main components of the developed Moodle activity module, the features available to teachers and students, and the resulting GitLab integration.

9.1. Overview

The plugins provide a GitLab Activity module to automate group management, GitLab resources provisioning and other labors. It also acts as the central point to access all the important informations. All the useful informations such as instructions, reviews, submission, starter code are available in a single GitLab repository per group and are all accessible within Moodle directly. Both platforms keep synchronized using bidirectional communication.

The developed solution provides a Moodle activity module that integrates GitLab into the management of practical assignments. It automates the creation and management of student groups, the provisioning of GitLab resources, and the configuration of the associated development workflow, significantly reducing the manual effort required from teachers.

The activity module acts as the central entry point for both teachers and students. All information related to an assignment—including the project description, starter code, repositories, submissions, reviews, and the reference solution—is accessible directly from Moodle.

Each student group is provided with a dedicated GitLab repository that serves as the collaborative workspace for the assignment. In addition, teachers have access to a private template repository used to prepare the practical work. This repository contains the starter code distributed to student groups as well as the reference solution used for evaluation and published to students after the submission deadline.

The integration maintains synchronization between Moodle and GitLab through bidirectional communication. Actions performed on either platform, such as repository submissions, or evaluations, are propagated automatically, ensuring that both systems remain consistent throughout the assignment lifecycle.

9.2. Teacher Features

The GitLab activity module provides teachers with a centralized interface for creating and managing assignments. When creating an activity, teachers can define the assignment properties, including its name, description, instructions, submission deadline, maximum group size, and the reviewers responsible for the evaluation.

For each activity, the module automatically provisions a dedicated GitLab group composed of a private template repository and one repository for each student group. The repository structure is organized consistently across all assignments, while repository permissions are configured automatically according to the assigned roles. Students receive access only to their group's repository, whereas reviewers are granted the permissions required to evaluate submissions.

The template repository serves as the reference for preparing the practical assignment. Teachers can maintain the starter code, assignment instructions, and reference solution from a single location. Any updates to the starter code or instructions can be propagated to existing group repositories, allowing corrections or improvements to be distributed even after student repositories have been created. Group modifications can also be reviewed from a merge request in the template repository,

giving access to all the submissions from a single repository. This reduces administrative overhead and simplifies the management of large numbers of student repositories.

The activity dashboard provides an overview of all student groups and their associated repositories. Teachers can create, modify, or remove groups, monitor participation, and access each group's repository directly from Moodle. The dashboard also centralizes submission information, allowing reviewers to determine whether submissions were made before or after the submission deadline, inspect the latest CI/CD pipeline results, review merge requests, retrieve the project source code for local evaluation, and identify groups whose submissions have already been graded.

Once the evaluation is complete, teachers can publish the reference solution and automatically notify students that their work has been reviewed.

9.3. Student Features

Students can create or join groups by accessing the list of available. Each group is provided with access to a dedicated GitLab repository, where members can collaborate on the assignment, manage their code, and submit their work. The practical work instructions and due date are also available in a GitLab issue.

Additionally, students are informed when modifications are made to protected files, helping them identify changes that may affect their implementation.

Students can also access assignment due dates through the Moodle calendar and receive reminder notifications to help them manage their workload and complete submissions in time.

Once grading is completed, students are notified on Moodle and can access feedback through GitLab merge requests, allowing them to associate evaluation comments directly with their submitted code.

Following the completion of the assignment, the reference solution is published and made available to students. This allows them to compare their implementation with the expected solution, better understand the intended approach, and identify possible improvements in their work.

9.4. Integration Flows

The integration relies on a set of automated workflows to provision GitLab resources and react to GitLab hooks. These workflows cover the main stages of the practical assignment lifecycle, from module initialization and group creation to repository synchronization and grading notifications.

The following sequence diagrams illustrate the main integration flows:

9.4.1. *Module creation*

The module creation flow describes the automated initialization of a practical assignment, including the creation of the GitLab group, template repository, configuration of permissions, protected files, webhooks, and Moodle scheduling tasks.

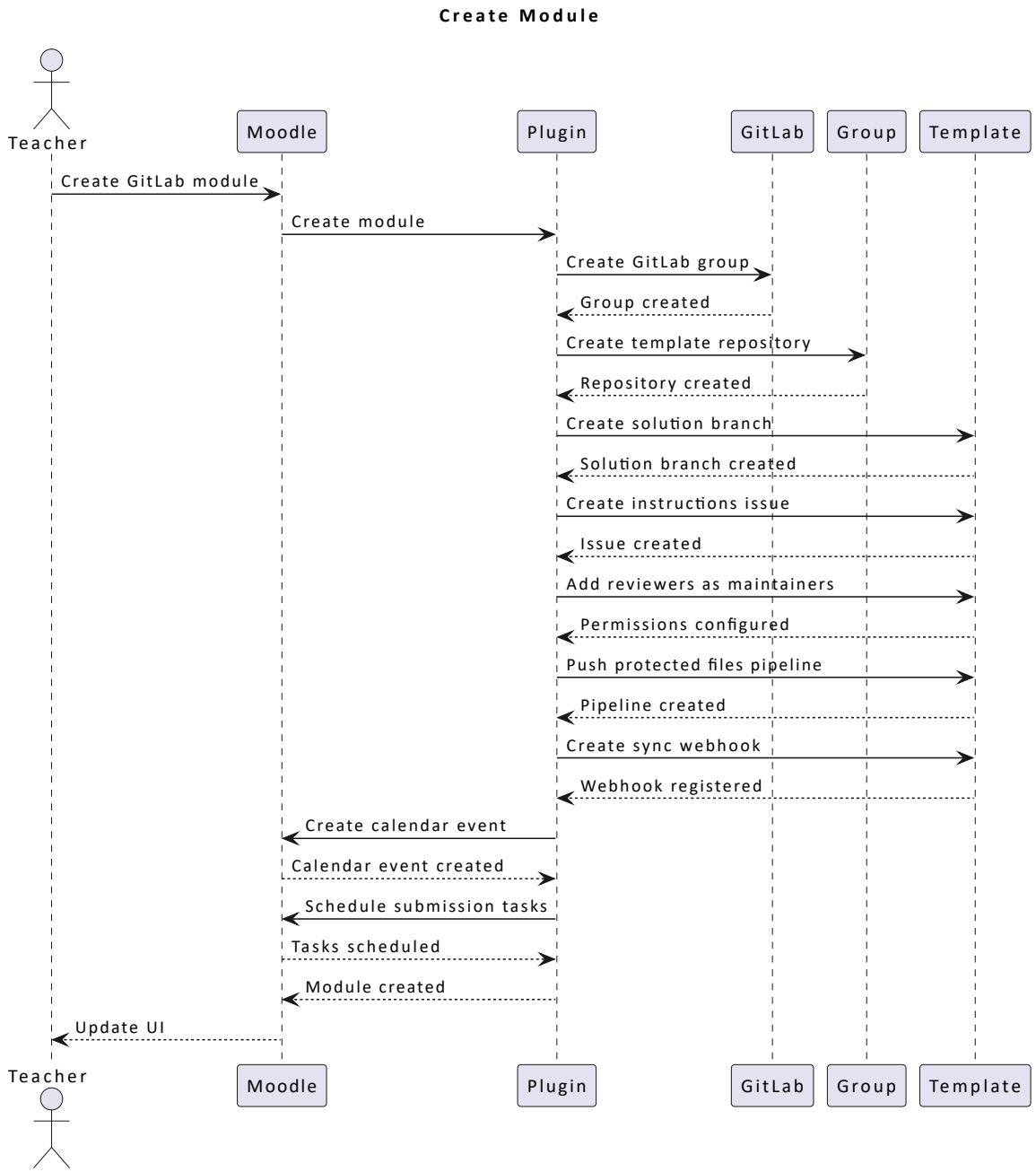

Figure 31: Module creation flow

9.4.2. *Group creation*

The group creation flow presents the provisioning of student workspaces, including repository forking, branch configuration, merge request setup, permission assignment, and synchronization of assignment instructions.

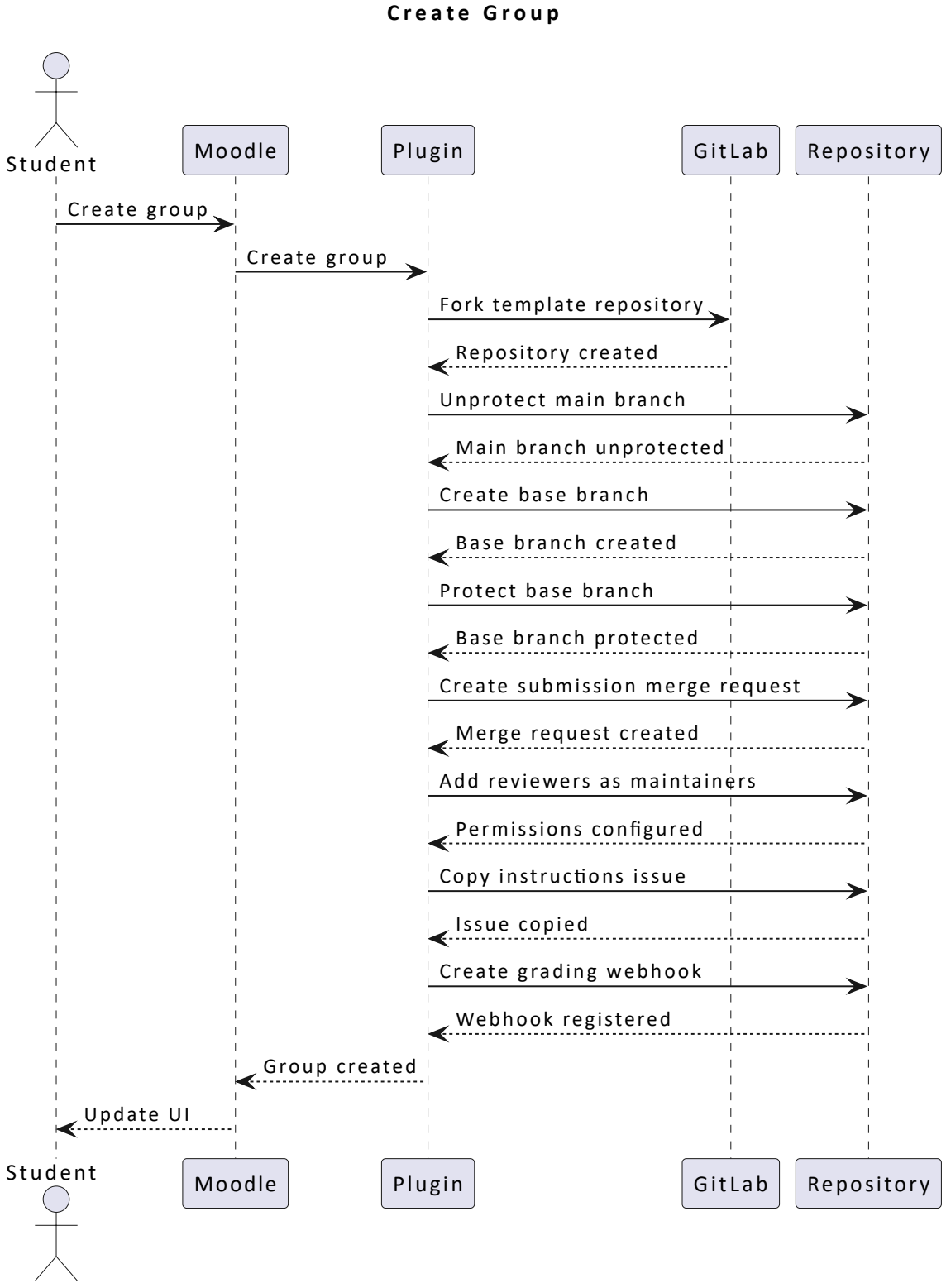

Figure 32: Group creation flow

9.4.3. *Template repository webhook*

The template repository webhook flow illustrates how updates made to the template repository are propagated to group repositories. It handles synchronization events such as code updates and instruction changes while avoiding duplicate merge requests.

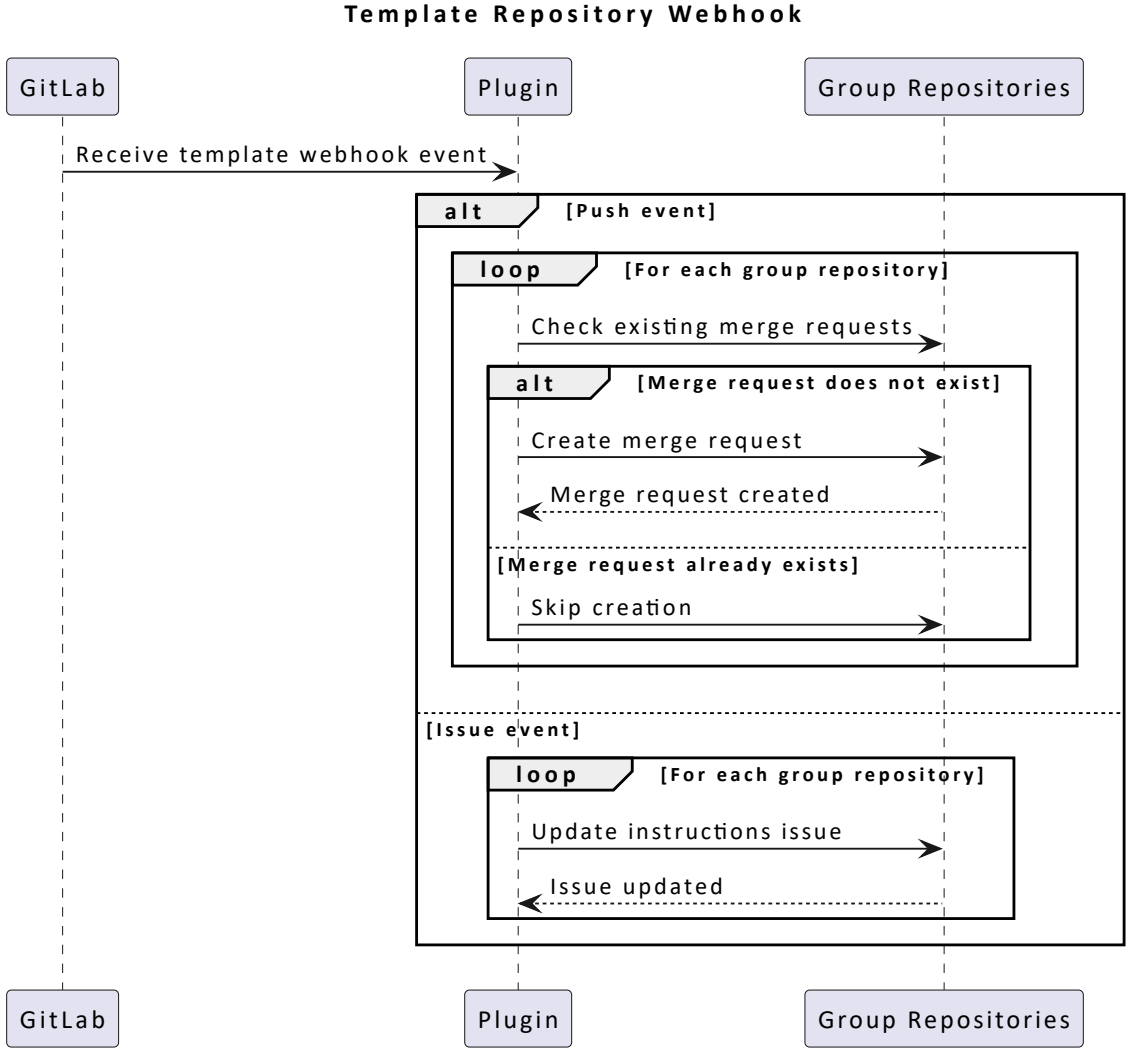

Figure 33: Template repository webhook flow

9.4.4. *Group repository webhook*

The group repository webhook flow describes the grading notification process triggered by merge request events. Once a submission is reviewed and the merge request is closed, notifications are sent to the corresponding group members.

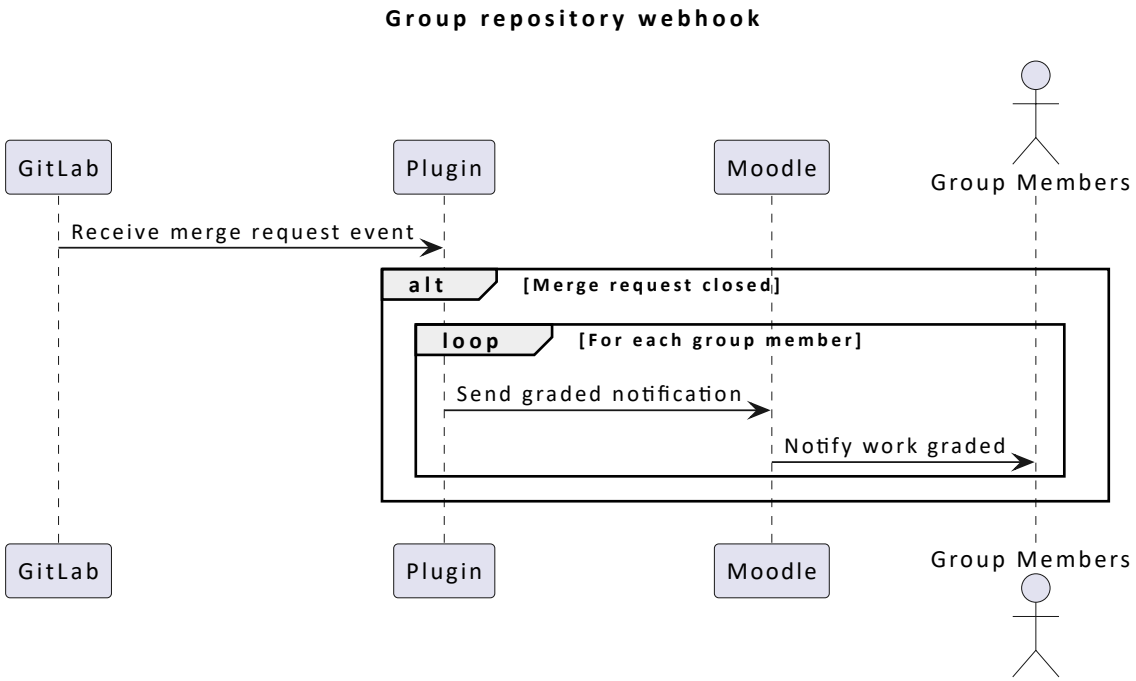

Figure 34: Group repository webhook flow

9.5. Result

The project achieved a seamless integration between Moodle and GitLab, providing a unified workflow for the creation, management, submission, and evaluation of practical assignments. By automating repository provisioning, synchronization, permission management, and feedback workflows, the system significantly reduces the amount of manual administration required from teachers. The integration acts as a bridge between the Learning Management System (LMS) and Version Control System (VCS), reducing platform fragmentation and establishing a reference workflow for managing practical assignments.

Overall, the solution standardizes practical assignment management and improves collaboration between students and teachers. It automates repetitive administrative tasks as much as possible while preserving the flexibility and customization required to adapt assignments to different teaching contexts and requirements.

Contribution and Future Work

10.1. Project Contribution

Before this project, practical assignment management often required manual coordination between Moodle and GitLab, including repository creation, permission configuration, assignment setup, and synchronization of updates. While existing solutions provided partial automation or integration capabilities, they generally lacked deep Learning Management System (LMS) integration or relied on proprietary platforms.

This project extends the existing workflow by introducing an open-source, Moodle-native integration that connects Moodle and GitLab into a unified assignment management process. The proposed solution automates repetitive administrative tasks while preserving the flexibility and customization required for different teaching contexts, avoiding the limitations often introduced by overly rigid automation systems. By reducing platform fragmentation and providing a reference workflow for practical assignment management, the solution improves the consistency and maintainability of computer science education practices.

10.2. Future Work

Several areas could be improved in future iterations of the project. First, additional work is required to strengthen error handling and improve user feedback. Providing clearer error messages and guidance would help users better understand the cause of failures and facilitate troubleshooting.

Another improvement area concerns Moodle permission management. Each action should have its required permissions explicitly defined and validated. Moodle provides a capability-based permission system that allows fine-grained control through custom capabilities, which should be integrated to ensure better security, and modularity of the plugin.

Finally, future development could focus on extending support to additional Version Control Systems (VCS), such as GitHub. Supporting multiple VCS platforms would increase the flexibility of the solution and allow institutions to adopt the solution within different technical environments.

Appendices

11.1. Planning

See gantt.xlsx

11.2. Weekly meeting reports

See reports.pdf

11.3. Source code

GitHub Repository [27]

11.4. Project documentation

Documentation [28]

11.5. Kanban Board

Kanban Board [19]

Bibliography

[1] Moodle, “Moodle.” Accessed: Jul. 16, 2026. [Online]. Available: https://moodle.org/

[2] Moodle, “Moodle architecture.” Accessed: Jul. 16, 2026. [Online]. Available: https://docs.moodle.org/dev/Moodle_architecture

[3] Instructure, “Canvas.” Accessed: Jul. 16, 2026. [Online]. Available: https://www.instructure.com/canvas

[4] PHP Group, “PHP.” Accessed: Jul. 16, 2026. [Online]. Available: https://www.php.net/

[5] GitLab, “GitLab.” Accessed: Jul. 16, 2026. [Online]. Available: https://about.gitlab.com/

[6] FHNW, “GitLab tools.” Accessed: Jul. 16, 2026. [Online]. Available: https://gitlab.fhnw.ch/gitlab-tools/gitlab-tools

[7] Stewart, R., “GitLab Haskell.” Accessed: Jul. 16, 2026. [Online]. Available: https://gitlab.com/robstewart57/gitlab-haskell

[8] python-gitlab, “Python GitLab documentation.” Accessed: Jul. 16, 2026. [Online]. Available: https://python-gitlab.readthedocs.io/

[9] GitLab PHP, “GitLab PHP API client.” Accessed: Jul. 16, 2026. [Online]. Available: https://github.com/GitLabPHP/Client

[10] GitHub, “GitHub Classroom transitioning to partners.” Accessed: Jul. 16, 2026. [Online]. Available: https://github.com/orgs/community/discussions/196615

[11] Codio, “Codio.” Accessed: Jul. 16, 2026. [Online]. Available: https://www.codio.com/

[12] Foundation 50, “Classroom 50.” Accessed: Jul. 16, 2026. [Online]. Available: https://github.com/foundation50/classroom50/wiki

[13] RepoBee, “RepoBee.” Accessed: Jul. 16, 2026. [Online]. Available: https://repobee.org/

[14] Classmoji, “Classmoji.” Accessed: Jul. 16, 2026. [Online]. Available: https://github.com/classmoji/classmoji

[15] L. Schauer, R. Stewart, and M. Maarek, “Integrating Canvas and GitLab to enrich learning processes,” in *2024 IEEE/ACM 46th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*, IEEE, 2024, pp. 180–190. doi: 10.1145/3639474.3640056.

[16] Moodle, “Encryption.” Accessed: Jul. 16, 2026. [Online]. Available: https://wimski.org/api/5.0/d4/d91/classcore_1_1encryption.html

[17] Gradescope, “Gradescope.” Accessed: Jul. 16, 2026. [Online]. Available: https://www.gradescope.com/

[18] ANS, “ANS.” Accessed: Jul. 16, 2026. [Online]. Available: https://www.ans.app/

[19] Jouve, L., “Kanban board.” Accessed: Jul. 16, 2026. [Online]. Available: https://github.com/users/LeonardJouve/projects/3

[20] Jouve, L., “Proof of concept.” Accessed: Jul. 16, 2026. [Online]. Available: https://github.com/LeonardJouve/moodle-gitlab-integration/pull/14

[21] Jouve, L., “Continuous deployment.” Accessed: Jul. 16, 2026. [Online]. Available: https://github.com/LeonardJouve/moodle-gitlab-integration/pull/16

[22] HashiCorp, “Terraform.” Accessed: Jul. 16, 2026. [Online]. Available: https://developer.hashicorp.com/terraform

[23] Red Hat, “Ansible documentation.” Accessed: Jul. 16, 2026. [Online]. Available: https://docs.ansible.com/

[24] Authentik, “Authentik.” Accessed: Jul. 16, 2026. [Online]. Available: https://goauthentik.io/

[25] Traefik Labs, “Traefik.” Accessed: Jul. 16, 2026. [Online]. Available: https://traefik.io/traefik

[26] Moodle, “Cron.” Accessed: Jul. 16, 2026. [Online]. Available: https://docs.moodle.org/502/en/Cron

[27] Jouve, L., “Repository.” Accessed: Jul. 16, 2026. [Online]. Available: https://github.com/LeonardJouve/moodle-gitlab-integration

[28] Jouve, L., “Project documentation.” Accessed: Jul. 16, 2026. [Online]. Available: https://github.com/LeonardJouve/moodle-gitlab-integration/blob/main/README.md